SIEMENS

TIA Portal Openness: API for automation of engineering workflows

Manual

21.00.00.00

12/2025

Legal information

Warning notice system

This manual contains notices you have to observe in order to ensure your personal safety, as well as to prevent damage to property. The notices referring to your personal safety are highlighted in the manual by a safety alert symbol, notices referring only to property damage have no safety alert symbol. These notices shown below are graded according to the degree of danger.

⚠ **DANGER**
indicates that death or severe personal injury **will** result if proper precautions are not taken.

⚠ **WARNING**
indicates that death or severe personal injury **may** result if proper precautions are not taken.

⚠ **CAUTION**
indicates that minor personal injury can result if proper precautions are not taken.

NOTICE
indicates that property damage can result if proper precautions are not taken.

If more than one degree of danger is present, the warning notice representing the highest degree of danger will be used. A notice warning of injury to persons with a safety alert symbol may also include a warning relating to property damage.

Qualified Personnel

The product/system described in this documentation may be operated only by **personnel qualified** for the specific task in accordance with the relevant documentation, in particular its warning notices and safety instructions. Qualified personnel are those who, based on their training and experience, are capable of identifying risks and avoiding potential hazards when working with these products/systems.

Proper use of Siemens products

Note the following:

⚠ **WARNING**
Siemens products may only be used for the applications described in the catalog and in the relevant technical documentation. If products and components from other manufacturers are used, these must be recommended or approved by Siemens. Proper transport, storage, installation, assembly, commissioning, operation and maintenance are required to ensure that the products operate safely and without any problems. The permissible ambient

conditions must be complied with. The information in the relevant documentation must be observed.

Trademarks

All names identified by ® are registered trademarks of Siemens Aktiengesellschaft. The remaining trademarks in this publication may be trademarks whose use by third parties for their own purposes could violate the rights of the owner.

Disclaimer of Liability

We have reviewed the contents of this publication to ensure consistency with the hardware and software described. Since variance cannot be precluded entirely, we cannot guarantee full consistency. However, the information in this publication is reviewed regularly and any necessary corrections are included in subsequent editions.

Siemens Aktiengesellschaft
Digital Industries
Postfach 48 48
90026 NÜRNBERG
GERMANY

12/2025 Subject to change

Copyright © Siemens 2024.
All rights reserved

Table of content

1 TIA Portal Openness: API for automation of engineering workflows 15
1.1 Cybersecurity information 15
1.2 What's new in TIA Portal Openness? 16
1.3 Basics 19
1.3.1 Installation 20
1.3.1.1 Requirements for TIA Portal Openness 20
1.3.1.2 Installing TIA Portal Openness 21
1.3.1.3 Adding users to the 'Siemens TIA Openness' user group 21
1.3.1.4 Using Windows Service 23
1.3.1.5 Accessing the TIA Portal 24
1.3.1.6 Configurations 24
1.3.2 Openness tasks 27
1.3.2.1 Introduction 27
1.3.2.2 Applications 28
1.3.2.3 Export/import 29
1.4 TIA Portal Openness API 29
1.4.1 TIA Portal Openness object 29
1.4.1.1 TIA Portal Openness object model 29
1.4.1.2 Blocks and types of the TIA Portal Openness object model 34
1.4.1.3 Hierarchy of hardware objects of the object model 39
1.4.1.4 Object list 41
1.4.2 General functions 47
1.4.2.1 Introduction 47
1.4.2.2 Information about installed TIA Portal Openness versions 48
1.4.2.3 TIA Portal Openness IntelliSense support 49
1.4.2.4 Standard libraries 49
1.4.2.5 Use of the code examples 51
1.4.2.6 Example program 54
1.4.2.7 Programming steps 0
1.4.2.8 Connecting to the TIA Portal 0
1.4.2.9 TIA Portal Openness firewall 0
1.4.2.10 Event handlers 0
1.4.2.11 Program-controlled acknowledgement of dialogs with system events 0
1.4.2.12 Terminating the connection to the TIA Portal 0
1.4.2.13 Diagnostic interfaces on TIA Portal 0
1.4.2.14 Exclusive access 0
1.4.2.15 Dynamic behaviours 0
1.4.2.16 Working with associations 0
1.4.2.17 Working with compositions 0

1.4.2.18 Verifying object equality 0
1.4.2.19 Read operations for attributes 0
1.4.2.20 Transaction handling 0
1.4.2.21 Creating a DirectoryInfo/FileInfo object 0
1.4.2.22 Self-description support for attributes, navigators, actions, and services 0
1.4.2.23 Setting attributes of Blocks, DBs & UDTs 0
1.4.2.24 Notes on performance of TIA Portal Openness 0
1.4.3 Functions for projects and project data 0
1.4.3.1 Opening a project 0
1.4.3.2 Creating a project 0
1.4.3.3 Accessing general settings of the TIA Portal 0
1.4.3.4 Archiving and Retrieving a project 0
1.4.3.5 Accessing read-only TIA Portal project 0
1.4.3.6 Accessing languages 0
1.4.3.7 Determining the object structure and attributes 0
1.4.3.8 Access software target 0
1.4.3.9 Accessing and enumerating multilingual texts 0
1.4.3.10 Updating project properties 0
1.4.3.11 Read project related attributes 0
1.4.3.12 Deleting project graphics 0
1.4.3.13 Compiling a project 0
1.4.3.14 Modifying project in UMAC 0
1.4.3.15 Saving a project 0
1.4.3.16 Closing a project 0
1.4.3.17 Export/Import system diagnostics settings 0
1.4.3.18 Identifying cross session object for Hardware & Software 0
1.4.3.19 Managing exception for missing component in project 0
1.4.3.20 Accessing multilingual support 0
1.4.4 Functions for accessing Teamcenter Gateway 0
1.4.4.1 Connecting to the Teamcenter Gateway 0
1.4.4.2 Disconnecting to the Teamcenter Gateway 0
1.4.4.3 Connecting to the ConnectSSO 0
1.4.4.4 Saving Project/Global library in Teamcenter 0
1.4.4.5 Saving Project/Global library with proxy object 0
1.4.4.6 Saving Project/Global library with new item 0
1.4.4.7 Accessing custom attributes in Teamcenter 0
1.4.4.8 Saving Project to Teamcenter as new item 0
1.4.4.9 Saving project using save to item with proxy object 0
1.4.4.10 Saving Project as new item with proxy object 0
1.4.4.11 Saving project using save as new revision with proxy object 0
1.4.4.12 Saving Project to Teamcenter as new revision 0
1.4.4.13 Downloading Project/Global library from Teamcenter 0
1.4.4.14 Searching Project / Global library from Teamcenter 0
1.4.4.15 Checkout Project/Global library dataset in Teamcenter 0
1.4.4.16 Checkin Project/Global library dataset in Teamcenter 0
1.4.4.17 Cancel checkout dataset in Teamcenter 0

1.4.5 Functions for Connections 0
1.4.5.1 Configurable attributes of a port-to-port connection 0
1.4.6 Functions on libraries 0
1.4.6.1 Functions for objects and instances 0
1.4.6.2 Accessing global libraries 0
1.4.6.3 Accessing library base type 0
1.4.6.4 Accessing global library languages 0
1.4.6.5 Opening libraries 0
1.4.6.6 Comparing libraries 0
1.4.6.7 Enumerating open libraries 0
1.4.6.8 Saving and closing libraries 0
1.4.6.9 Archiving and retrieving a library 0
1.4.6.10 Creating global libraries 0
1.4.6.11 Accessing folders in a library 0
1.4.6.12 Accessing types 0
1.4.6.13 Accessing type versions 0
1.4.6.14 Accessing blocks located in library 0
1.4.6.15 Accessing instances 0
1.4.6.16 Accessing master copies 0
1.4.6.17 Create master copy from a project in library 0
1.4.6.18 Cleaning up library 0
1.4.6.19 Create an object from a master copy 0
1.4.6.20 Copying master copies 0
1.4.6.21 Creating new type from document 0
1.4.6.22 Updating type from document 0
1.4.6.23 Determining out-of-date type instances 0
1.4.6.24 Updating the project 0
1.4.6.25 Updating a library 0
1.4.6.26 Cleaning up library 0
1.4.6.27 Harmonize project from project library 0
1.4.6.28 Updating Structure during Library Update 0
1.4.6.29 Updating libraries type 0
1.4.6.30 Deleting library content 0
1.4.6.31 Setting default version of a type 0
1.4.6.32 Getting default version of a type 0
1.4.6.33 Improved types consistency state 0
1.4.6.34 Editing library type version 0
1.4.6.35 Discarding library type version 0
1.4.6.36 Managing alarm text list in PLC and Mastercopy library 0
1.4.6.37 Managing DoNotUse in Library 0
1.4.6.38 Exporting library type version as document 0
1.4.6.39 Retrieving supported export formats for library 0
1.4.6.40 Releasing library type version 0
1.4.7 Functions for accessing devices, networks and connections 0
1.4.7.1 Open the 'Devices & networks' editor 0
1.4.7.2 Querying PLC and HMI targets 0

1.4.7.3 Accessing attributes of an address object 0
1.4.7.4 Accessing the channels of a module 0
1.4.8 Functions on networks 0
1.4.8.1 Creating a subnet 0
1.4.8.2 Accessing subnets 0
1.4.8.3 Accessing internal subnets 0
1.4.8.4 Get type identifier of subnets 0
1.4.8.5 Accessing attributes of a subnet 0
1.4.8.6 Deleting a global subnet 0
1.4.8.7 Enumerate all participants of a subnet 0
1.4.8.8 Enumerate IO systems of a subnet 0
1.4.8.9 Accessing nodes 0
1.4.8.10 Accessing attributes of a node 0
1.4.8.11 Connecting a node to a subnet 0
1.4.8.12 Disconnect a node from a subnet 0
1.4.8.13 Creating an IO system 0
1.4.8.14 Accessing the attributes of an IO system 0
1.4.8.15 Connecting an IO connector to an IO system 0
1.4.8.16 Get master system or IO system of an interface 0
1.4.8.17 Get an IO Controller 0
1.4.8.18 Get an IO Connector 0
1.4.8.19 Disconnecting an IO connector from an IO system or a DP mastersystem 0
1.4.8.20 Accessing attributes of a DP mastersystem 0
1.4.8.21 Accessing attributes of PN Driver 0
1.4.8.22 Accessing attributes of a profinet io system 0
1.4.8.23 Deleting a DP master system 0
1.4.8.24 Deleting a profinet io system 0
1.4.8.25 Creating a DP master system 0
1.4.8.26 Accessing port interconnection information of port device item 0
1.4.8.27 Attributes of port inter-connection 0
1.4.8.28 Accessing the attributes of a port 0
1.4.8.29 Enumerate DP master systems of a subnet 0
1.4.8.30 Enumerate assigned IO connectors 0
1.4.8.31 Connecting a DP IO connector to a DP master system 0
1.4.8.32 Accessing AS-i profile and parameter attributes for virtual slaves 0
1.4.8.33 Accessing CM DP as DP slave and transfer area 0
1.4.8.34 Coupling 1516 isochron central and decentral Profinet 0
1.4.8.35 Openness for CP 1604/CP 1616/CP 1626 0
1.4.8.36 Openness transfer areas for PnPn coupler 0
1.4.8.37 Openness virtual modules/submodules for ET 200SP PN HF 0
1.4.8.38 Accessing domain settings 0
1.4.8.39 Accessing redundancy mode attribute 0
1.4.8.40 Creating CCDX transfer area 0
1.4.8.41 Deleting CCDX transfer area 0
1.4.8.42 Modifying CCDX transfer areas configuration 0
1.4.9 Functions on devices 0

1.4.9.1 Mandatory attributes of devices 0
1.4.9.2 Get type identifier of devices and device items 0
1.4.9.3 Set App ID on device and device Items 0
1.4.9.4 Get App ID on device and device Items 0
1.4.9.5 Remove App ID on device and device items 0
1.4.9.6 Creating a device 0
1.4.9.7 Enumerating devices 0
1.4.9.8 Accessing devices 0
1.4.9.9 Accessing the TIA Portal hardware catalog 0
1.4.9.10 Deleting a device 0
1.4.9.11 Bulk changing hardware parameters 0
1.4.10 Functions on device items 0
1.4.10.1 Mandatory attributes of device items 0
1.4.10.2 Creating and plugging a device item 0
1.4.10.3 Moving device items into another slot 0
1.4.10.4 Copying a device item 0
1.4.10.5 Setting user defined logo settings 0
1.4.10.6 Deleting a device item 0
1.4.10.7 Enumerate device items 0
1.4.10.8 Accessing device items 0
1.4.10.9 Accessing device item as interface 0
1.4.10.10 Accessing change device 0
1.4.10.11 Accessing attributes of an I/O device interface 0
1.4.10.12 Getting subnet of device item 0
1.4.10.13 Accessing attributes of IoController 0
1.4.10.14 Accessing attributes of IoConnector 0
1.4.10.15 Accessing address controller 0
1.4.10.16 Accessing addresses 0
1.4.10.17 Accessing hardware identifiers 0
1.4.10.18 Accessing hardware identifier controller 0
1.4.10.19 Accessing channels of device items 0
1.4.10.20 Accessing normalized type identifiers 0
1.4.10.21 Creating and exporting psc file 0
1.4.10.22 Connection handling for extension racks 0
1.4.10.23 Telecontrol management datapoint 0
1.4.11 Functions for accessing the data of a PLC device 0
1.4.11.1 Functions for downloading data to PLC device 0
1.4.11.2 Functions for accessing PLC service 0
1.4.11.3 Blocks 0
1.4.11.4 Tags and Tag tables 0
1.4.11.5 Functions for software units 0
1.4.11.6 Functions for fail-safe unit 0
1.4.12 Functions for accessing the data of an HMI device 0
1.4.12.1 Screens 0
1.4.12.2 Cycles 0
1.4.12.3 Text lists 0

1.4.12.4 Graphic lists 0
1.4.12.5 Connections 0
1.4.12.6 Tag table 0
1.4.12.7 VB scripts 0
1.4.13 Functions for Version Control Interface 0
1.4.13.1 Accessing VCI system group in project 0
1.4.13.2 Enumerating user groups in VCI group 0
1.4.13.3 Accessing VCI Workspace with library properties 0
1.4.13.4 Enumerating VCI mapped objects in VCI workspace 0
1.4.13.5 Creating VCI user group in VCI group 0
1.4.13.6 Connecting object 0
1.4.13.7 Updating VCI group properties 0
1.4.13.8 Deleting VCI user group 0
1.4.13.9 Enumerating VCI workspaces in VCI group 0
1.4.13.10 Creating VCI workspace in VCI group 0
1.4.13.11 Updating VCI workspace properties 0
1.4.13.12 Deleting VCI workspace 0
1.4.13.13 Exporting individual object to workspace 0
1.4.13.14 Getting supported file formats 0
1.4.13.15 Getting VCI workspace mapping properties 0
1.4.13.16 Deleting mapped object 0
1.4.13.17 Synchronizing mapped object 0
1.4.13.18 Getting status of mapped object 0
1.4.13.19 Getting child status of a mapped hierarchical object 0
1.4.13.20 Accessing synchronization status of child objects 0
1.4.13.21 Creating VCI workspace with workspace language 0
1.4.13.22 Initializing VCI object status 0
1.4.13.23 Importing Simatic ML with inactive culture 0
1.4.14 Functions support for Multiuser 0
1.4.14.1 Creating project server connection 0
1.4.14.2 Adding project to server 0
1.4.14.3 Editing project server connection 0
1.4.14.4 Deleting project server connection 0
1.4.14.5 Getting project server connection from TIA Portal Setting 0
1.4.14.6 Getting project server group 0
1.4.14.7 Adding a project to server 0
1.4.14.8 Getting server projects 0
1.4.14.9 Getting available local sessions 0
1.4.14.10 Creating a local session 0
1.4.14.11 Saving a local session 0
1.4.14.12 Deleting local session from server 0
1.4.14.13 Opening local/exclusive session 0
1.4.14.14 Opening server project 0
1.4.14.15 Checking marked objects 0
1.4.14.16 Closing local session 0
1.4.14.17 Commiting server project changes 0

1.4.14.18 Getting lockstate provider 0
1.4.14.19 Checking project locked 0
1.4.14.20 Getting lock owner 0
1.4.14.21 Checking session UptoDate 0
1.4.14.22 Getting markings from local session 0
1.4.15 Functions for accessing data of the user management 0
1.4.15.1 Functions for accessing user, roles and function rights 0
1.4.15.2 Functions for UMAC Global Users and UMC Server 0
1.4.16 Functions on OPC 0
1.4.16.1 Configuring OPC UA server secure communication protocol 0
1.4.16.2 Setting OPC UA security policy 0
1.4.16.3 Accessing OPC UA reference namespace 0
1.4.16.4 Accessing OPC UA server (SIMATIC) interface 0
1.4.16.5 Accessing OPC UA server interface 0
1.4.17 Functions for SIMATIC Drive Controller 0
1.4.17.1 Accessing PLC and X142 channel properties 0
1.4.17.2 Accessing PROFIdrive Integrated properties 0
1.4.17.3 Creating SINAMICS Integrated 0
1.4.17.4 Getting address of central module (X142) 0
1.4.17.5 Getting address of drive controller telegram 0
1.4.18 Exceptions 0
1.4.18.1 Handling exceptions 0
1.4.19 F-related Openness 0
1.4.19.1 F-related Openness 0
1.4.19.2 SafetyBaseIdProvider 0
1.4.19.3 SafetyModificationsPossible 0
1.4.19.4 UsernameForFChangeHistory 0
1.4.19.5 F-Signatures 0
1.4.19.6 SafetySignatureProvider 0
1.4.19.7 SafetyAdministration 0
1.4.19.8 SafetyPrintout 0
1.4.19.9 Hardware attributes 0
1.4.20 Functions for Safety Validation Assistant 0
1.4.20.1 Accessing Safety Validation Assistant 0
1.4.20.2 Querying the Activation Tests 0
1.4.20.3 Querying the Evaluation Devices 0
1.4.20.4 Creating new Activation Tests 0
1.4.20.5 Duplicating Activation Tests 0
1.4.20.6 Copying Activation Tests from a library 0
1.4.20.7 Querying Available Devices of the Activation Test 0
1.4.20.8 Modifying Activation Tests 0
1.4.20.9 Overall state of the Activation Tests 0
1.4.20.10 Validity of the Activation Tests 0
1.4.20.11 Generating Activation Test reports 0
1.4.20.12 Exporting Activation Tests 0
1.4.20.13 Importing Activation Tests 0

1.4.20.14 Querying the Safety Functions 0
1.4.20.15 Creating new Safety Functions 0
1.4.20.16 Duplicating Safety Functions 0
1.4.20.17 Resetting Test Results of Safety Functions 0
1.4.20.18 Modifying Safety Functions 0
1.4.20.19 State of the Safety Functions 0
1.4.20.20 Validity of the Safety Functions 0
1.4.20.21 Exporting Safety Functions 0
1.4.20.22 Importing Safety Functions 0
1.4.20.23 Querying the Conditions 0
1.4.20.24 Creating new Conditions 0
1.4.20.25 Validity of the Conditions 0
1.4.20.26 Getting the Trace Configuration 0
1.4.20.27 Modifying Trace Configuration 0
1.4.20.28 Validity of the Trace Configuration 0
1.5 Export/import 0
1.5.1 Overview 0
1.5.1.1 Basic principles of importing/exporting 0
1.5.1.2 Field of application for Import/Export 0
1.5.1.3 Version Specific Simatic ML Import 0
1.5.1.4 Editing the XML file 0
1.5.1.5 Exporting configuration data 0
1.5.1.6 Importing configuration data 0
1.5.2 Import/export of project data 0
1.5.2.1 Project graphics 0
1.5.2.2 Project texts 0
1.5.3 Importing/exporting data of an HMI device 0
1.5.3.1 Importing/exporting data of an HMI device 0
1.5.3.2 Data structure for import/export 0
1.5.3.3 Tag tables 0
1.5.3.4 VB scripts 0
1.5.3.5 Text lists 0
1.5.3.6 Graphic lists 0
1.5.3.7 Connections 0
1.5.3.8 Screens 0
1.5.4 Importing/exporting data of a PLC device 0
1.5.4.1 Blocks 0
1.5.4.2 Tag tables 0
1.5.5 Importing/exporting hardware data 0
1.5.5.1 AML file format 0
1.5.5.2 Pruned AML 0
1.5.5.3 Overview of the objects and parameters of the CAx import/export 0
1.5.5.4 Structure of the CAx data for importing/exporting 0
1.5.5.5 AML type identifiers 0
1.5.5.6 Export/Import of base unit Information via AML 0
1.5.5.7 Export/Import AML with extension rack connection 0

1.5.5.8 Export/import AML file with GSD/GSDML custom attributes 0
1.5.5.9 Export/Import AML file with non-GSD/GSDML custom attributes 0
1.5.5.10 Export/import AML file with pluggable port configuration 0
1.5.5.11 Export/Import AML file with IO Link 0
1.5.5.12 Connection handling for extension racks 0
1.5.5.13 Export/Import AML file with complex tag configuration 0
1.5.5.14 Export of CAx data 0
1.5.5.15 Import of CAx data 0
1.5.5.16 Export/Import of sub modules 0
1.5.5.17 Export/Import AML file in UMAC environment 0
1.5.5.18 Export/Import AML file with normalized type identifier 0
1.5.5.19 Import CAx data without logical address 0
1.5.5.20 Exceptions during import and export of CAx data 0
1.5.5.21 Round trip exchange of devices and modules 0
1.5.5.22 Export/Import topology 0
1.5.5.23 Import of device with library references 0
1.5.5.24 Export of a device object 0
1.5.5.25 Import of a device object 0
1.5.5.26 Export/Import of device with set address 0
1.5.5.27 Export/Import of device with channels 0
1.5.5.28 Export of device item objects 0
1.5.5.29 Import of device item objects 0
1.5.5.30 Export/Import of GSD/GSDML based devices and device items 0
1.5.5.31 Export/Import device configuration with virtual interface 0
1.5.5.32 Export/Import of subnets 0
1.5.5.33 Export/Import of IO-systems 0
1.5.5.34 Export/Import of multilingual comments 0
1.5.5.35 Export/Import ET200 SP/ET200 AL devices with extension rack connections 0
1.5.5.36 Export/Import of PLC tags 0
1.5.5.37 Export/Import of RH/PLC 0
1.5.5.38 Export/Import AML with customized tag and deviceitems 0
1.5.5.39 Export/Import of FailSafe PLC 0
1.5.5.40 Export/Import of Failsafe IO 0
1.5.5.41 Export/Import vendor specific attribute 0
1.5.5.42 AML attributes versus TIA Portal Openness attributes 0
1.6 Major Changes 0
1.6.1 Major changes for long-term stability in TIA Portal Openness V20 0
1.6.2 Major changes for long-term stability in TIA Portal Openness V19 0
1.6.3 Major changes for long-term stability in TIA Portal Openness V18 0
1.6.4 Major changes for long-term stability in TIA Portal Openness V17 0
1.6.5 Major changes in TIA Portal Openness V16 0
1.6.6 Major changes in TIA Portal Openness V15.1 0
1.6.7 Major changes in TIA Portal Openness V15 0
1.6.8 Major changes in V14 SP1 0
1.6.8.1 Major changes in V14 SP1 0
1.6.8.2 Major changes in the object model 0

1.6.8.3 Changes on pilot functionality 0
1.6.8.4 Changes for export and import 0
1.6.9 Major changes in V14 0
1.6.9.1 Major changes of the object model 0
1.6.9.2 Before updating an application to TIA Portal Openness V14 0
1.6.9.3 Major string changes 0
1.6.9.4 Import of files generated with TIA Portal Openness V13 SP1 and previous 0

TIA Portal Openness: API for automation of engineering workflows

1

This section contains information on the following topics:

- Cybersecurity information
- Readme TIA Portal Openness
- What's new in TIA Portal Openness?
- Basics
- TIA Portal Openness API
- Export/import
- Major Changes

1.1 Cybersecurity information

Cybersecurity information

Siemens provides products and solutions with industrial cybersecurity functions that support the secure operation of plants, systems, machines and networks.

In order to protect plants, systems, machines and networks against cyber threats, it is necessary to implement – and continuously maintain – a holistic, state-of-the-art industrial cybersecurity concept. Siemens' products and solutions only form one element of such a concept.

Customers are responsible for preventing unauthorized access to their plants, systems, machines and networks. Such systems, machines and components should only be connected to an enterprise network or the internet if and to the extent such a connection is necessary and only when appropriate security measures (e.g. firewalls and/or network segmentation) are in place.

For additional information on industrial cybersecurity measures that may be implemented, please visit

https://www.siemens.com/global/en/products/automation/topic-areas/industrial-cybersecurity.html

Siemens' products and solutions undergo continuous development to make them more secure. Siemens strongly recommends that product updates are applied as soon as they are available and that the latest product versions are used. Use of product versions that are no longer supported, and failure to apply the latest updates may increase customer's exposure to cyber threats.

To stay informed about product updates, subscribe to the Siemens Industrial Cybersecurity RSS Feed under

https://new.siemens.com/global/en/products/services/cert.html

1.2 What's new in TIA Portal Openness?

Compatibility and long term stability

Siemens.Engineering libraries

The TIA Portal Openness architecture V21 has been extensively redesigned for seamless integration and long-term stability. The result is new modular TIA Portal Openness libraries with simplified assembly management.

Because the Siemens.Engineering libraries of V17, V18, V19, V20 are not included in the scope of delivery, applications and Add-Ins based on V17, V18, V19, V20 must be adapted to run in TIA Portal V21.

The new Siemens.Engineering libraries can be found in the installation directory under "PublicAPI\V21\net48\". For example, the V21 base library can be found in "C:\Program Files\Siemens\Automation\Portal V21\PublicAPI\V21\net48\Siemens.Engineering.Base.dll".

See also: Major changes for long-term stability in TIA Portal Openness V21

Exporting SimaticML files

The Siemens.Engineering libraries in the TIA Portal V21 installation will create SimaticML files of engineering version V21.

Importing SimaticML files

The Siemens.Engineering libraries in the TIA Portal V21 installation can import SimaticML files from engineering versions V18, V19, V20, and V21.

New features

The following new features and innovations are available in TIA Portal Openness V21. For more details on the various topics, refer to the relevant sections of the product documentation.

- General
  - TIA Portal Openness is no longer a setup option but an inherent feature of TIA Portal
  - Improved setup and update performance of products and options
  - Extensive redesign of the TIA Portal Openness architecture with new modular Openness libraries; see also: Major changes for long-term stability in TIA Portal Openness V21
  - Simplified assembly resolving dependent assemblies
  - Version-independent AllowList for the TIA Portal Openness firewall
  - Extended self-description support of APIs
  - Improved formatting of SimaticML
- TIA Portal Add-Ins

  - New modular Openness libraries; see also: Major changes for long-term stability in TIA Portal Openness V21
  - New types of TIA Portal Add-Ins: Corporate Add-Ins, System Add-Ins, and User Add-Ins
  - Create shared .NET class libraries for Openness and Add-Ins with uniformed Openness libraries
  - Possibility to get and attach to other TIA Portal processes
- Projects
  - Device specific export of project texts with category filter
- Libraries
  - Use master copies and library types for PLC documents such as Named value types (NVT)
- Hardware configuration
  - Create and manage communication connections (FDL, HMI, ISO, ISO-on-TCP, PtP, S7, TCP, UDP)
  - Features for hardware configuration in run (CiR)
  - Export PN-GSD file of an I-Device
  - Manage CP / TIM TeleControl data points
  - Configure project integrated shared device
  - Query status of GSDX signature for GSD devices
- Security
  - Support additional confidential configuration data protection settings of a PLC
- Hardware modules
  - Hardware parameter access for S7-1200 G2 (Standard)
  - Bulk operation support for Safety module parameters: ET200 SP Safety, ET200 AL Safety, ET200pro Safety, ET200eco PN Safety, ET200MP Safety
  - Access to further hardware parameters of SCALANCE
  - Resource configuration and export of configuration file for S7-1500 Software Controller (V30.0)
- CAx data exchange
  - Release of CAx Publication Tools V4.0 for hardware data management compatible with TIA Portal V21
- Online scenarios
  - Verify certificate details of a secure connection in online event handlers
  - Check secure communication state for passwords in online event handlers
  - Go online on a protected PLC
  - Go online on a PLC by specifying the IP address

  - Go online on a primary or backup R/H PLC by specifying the IP address
  - Speedup of reading online fingerprints of a PLC for quick station compare
  - Improved error message in case of issues with a secure connection
  - Download additional user files to PLC
- Station Upload
  - Continue upload device as new station for a PLC if a PROFINET IO device cannot be uploaded
- PLC user programs
  - Extended new text-based file exchange format for PLC blocks for programming language LAD, Safety-LAD, FBD, Safety-FBD, SCL and blocks with mixed languages as well as data blocks and PLC data types
  - Create snapshot of actual values for a data block
  - Load start values as actual values for a data block
  - Load snapshot as actual values for a data block
  - Read and write further block attributes such as for information, compilation, and download
  - Rename groups and objects in PLC software
  - Manage write protection of blocks
  - Access multilingual title and comment of objects in PLC software
  - Get PLC tags of a specific device item
- Safety engineering
  - PROFIsafe addressing capability F-BaseID
  - Read F-signatures from Safety Administration Editor
  - Enable and disable F-activation of an F-PLC
- Safety Validation Assistant
  - Create and manage activation tests
- Technology objects
  - Configure technology object SIMATIC Ident
  - Connecting with Superimposing Axes
  - Data block member and interpreter mapping for motion technology objects
- WinCC Unified
  - Create and manage HMI graphic lists
  - Access additional HMI runtime settings
  - New HMI controls: Alarm Line, Process Diagnostic Criteria Analysis, Process Diagnosis Graph Overview, Process Diagnosis Overview, Process Diagnosis PLC Code Viewer

  - New HMI widgets: Alarm Indicator
  - Export and import layout fields of a screen
- Alarming and ProDiag
  - Export and import supervision names and user labels
- SIMATIC Visualization Architect (SiVArc)
  - Configure acquisition mode
  - Configure layout data
  - Upgrade SiVArc definitions
  - Access tag members
- Web server
  - Manage default web pages and web applications
- OPC UA
  - Generate OPC UA nodes during creation of reference namespaces
  - Rename existing OPC UA reference namespace
  - Configure OPCA UA access control for users and roles
  - Dynamic management of OPC UA client certificates
- Version Control Interface (VCI)
  - Support of F-compliant PLC data types, FBD/LAD/SCL and mixed blocks and data blocks in SIMATIC SD format
  - Support of types in the project library: PLC blocks, PLC data types
- SINAMICS Startdrive
  - Access drive item hardware modules and their component names
  - Upload drive parameters

See also

Major changes for long-term stability in TIA Portal Openness V21

See also

Major changes in TIA Portal Openness V16
Major changes for long-term stability in TIA Portal Openness V17
TIA Portal Openness object model
Major changes in TIA Portal Openness V15.1
Major changes in TIA Portal Openness V15
Major changes in V14 SP1
Major changes in V14

1.3 Basics

1.3.1 Installation

1.3.1.1 Requirements for TIA Portal Openness

Requirements for using TIA Openness applications

- A product based on the TIA Portal is installed on the PC, for example, "STEP 7 Professional" or "WinCC Professional".
- The "TIA Portal Openness" is installed on the PC.

  See Installing TIA Portal Openness

Supported Windows operating systems

The following table shows which combinations of Windows operating system, TIA Portal and user application are mutually compatible:

Windows operating system	TIA Portal	User application
64-bit	64-bit	32-bit, 64-bit and "Any CPU"

Requirements for programming TIA Portal Openness applications

- Microsoft Visual Studio 2017 or later with .Net Framework SDK 4.8 and the Windows Classic Desktop package

Necessary user knowledge

- Knowledge as a system engineer
- Advanced knowledge of Microsoft Visual Studio 2017 or later with .Net Framework SDK 4.8
- Advanced knowledge of C# / VB.net and .Net Framework
- User knowledge of the TIA Portal

TIA Portal Openness remoting channels

The TIA Portal Openness remoting channels are registered as type IpcChannel with the “ensureSecurity” parameter set to "false".

Note

You should avoid registering another IpcChannel using a “ensureSecurity” parameter value other than "false" with a priority higher than or equal to “1”.

The IpcChannel is defined with the following attributes:

Attribute	Settings
"name" and "portName"	Set to $”{Process.Name}_{Process.Id}” or $”{Process.Name}_{Process.Id}_{AppDomain.Id}” when registered in an AppDomain other than the application’s default.
“priority”	Set with the default value of “1”.
“typeFilterLevel”	Set to “Full”.
“authorizedGroup”	Set to the NTAccount value string for the built-in user account (i.e. everyone).

See also

Adding users to the "Siemens TIA Openness" user group

1.3.1.2 Installing TIA Portal Openness

Introduction

TIA Portal Openness is an inherent feature of TIA Portal and is automatically installed.

Result

Depending on the different TIA Portal products and options, additional modular Siemens.Engineering assemblies will be installed.

A local Windows group "Siemens TIA Openness" is being created once during setup.

Note

Administrators must grant individual users the execution of any TIA Portal Openness application in general.

See also: Adding users to the "Siemens TIA Openness" user group

1.3.1.3 Adding users to the 'Siemens TIA Openness' user group

Introduction

When you install TIA Portal on your computer, the local Windows group "Siemens TIA Openness" is automatically created.

Whenever you try to access a TIA Portal process with your TIA Portal Openness application, TIA Portal verifies that you are a member of the "Siemens TIA Openness" user group, either directly or indirectly by way of another user group. If you are a member of the "Siemens TIA Openness" user group, the TIA Portal Openness application starts and establishes a connection to the TIA Portal instance, else an EngineeringSecurityException will be thrown telling you can’t access this process.

Note

This group membership permits you to execute any TIA Portal Openness application.

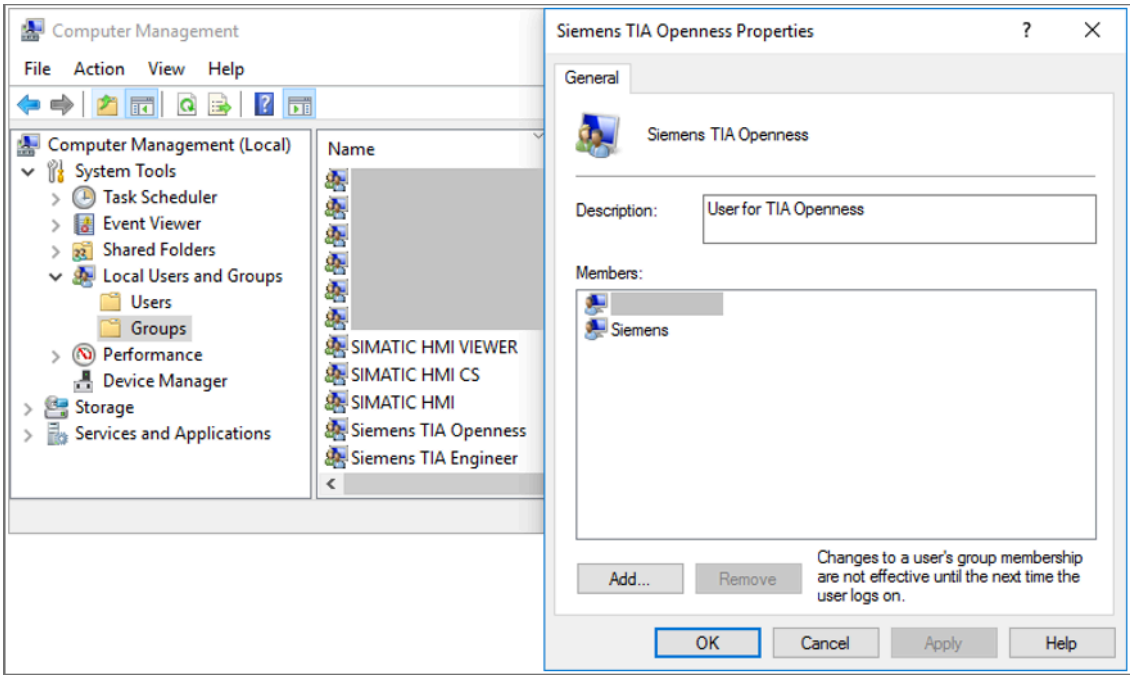
See also: TIA Portal Openness firewall

Procedure

Your administrator must manage the local group membership for "Siemens TIA Openness" in the local Microsoft Windows control panel.

1. Start the local “Computer Management” as administrator from the search in the Windows start menu
2. Go to “Groups” in “Local Users and Groups” to list the local user groups
3. Select “Siemens TIA Openness” and show the properties
4. Click on “Add...” to add a user
5. The affected user must fully sign out and sign in again

Example:



1.3.1.4 Using Windows Service

Introduction

Windows Services are long-running executables that run without a user interface and are installed using an installation utility. It is possible to develop such a Windows service application that uses TIA Portal Openness.

To do so, it is necessary to create a project of type "Windows Service (.NET Framework)" in Microsoft Visual Studio.

Requirement

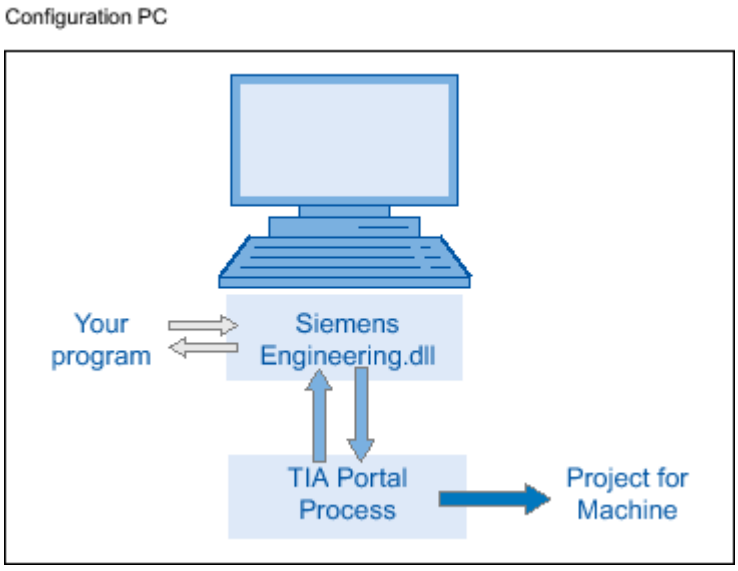
- The process executable of the Windows Service application needs to be added to the AllowList beforehand. This can be done by creating the Windows registry entries on installation of the Windows Service, for example as part of a setup created with a Windows Installer project.
  For more information about the AllowList, see section "Addition of an AllowList entry by connecting to TIA Portal" in chapter TIA Portal Openness firewall
- It is not possible to access running TIA Portal processes or to start TIA Portal with graphical interface. Only a new TIA Portal instance without user interface can be created.
- It is highly recommended to run the Windows Service with credentials of a dedicated Windows user account. It is possible but not recommended to use the local service account or the network service account. The local system account is not supported due to security restrictions and will always throw a EngineeringSecurityException.
- The used Windows account (either service or user) needs to be part of the local Windows user group "Siemens TIA Openness".

See also

TIA Portal Openness firewall

1.3.1.5 Accessing the TIA Portal

Overview



Procedure

1. Set up the development environment to access and start the TIA Portal.
2. Instantiate the object of the portal application in your program to start the portal.
3. Find the desired project and open it.
4. Access the project data.
5. Close the project and exit the TIA Portal.

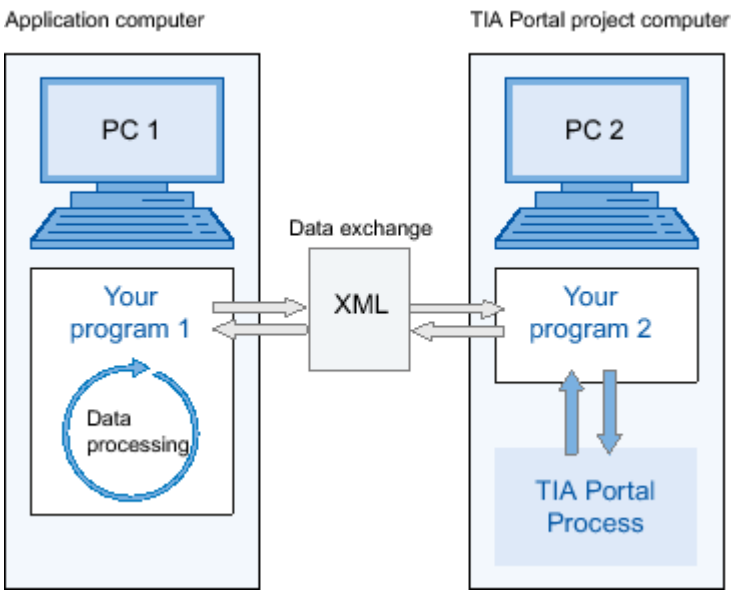
See also

Connecting to the TIA Portal
Terminating the connection to the TIA Portal

1.3.1.6 Configurations

You can work with two variants of the TIA Portal Openness:

Application and the TIA Portal are on different computers



- Data exchange takes place by XML files. The XML files can be exported or imported by your programs.
- The data exported from the TIA Portal project to PC2 can be modified on PC1 and re-imported.

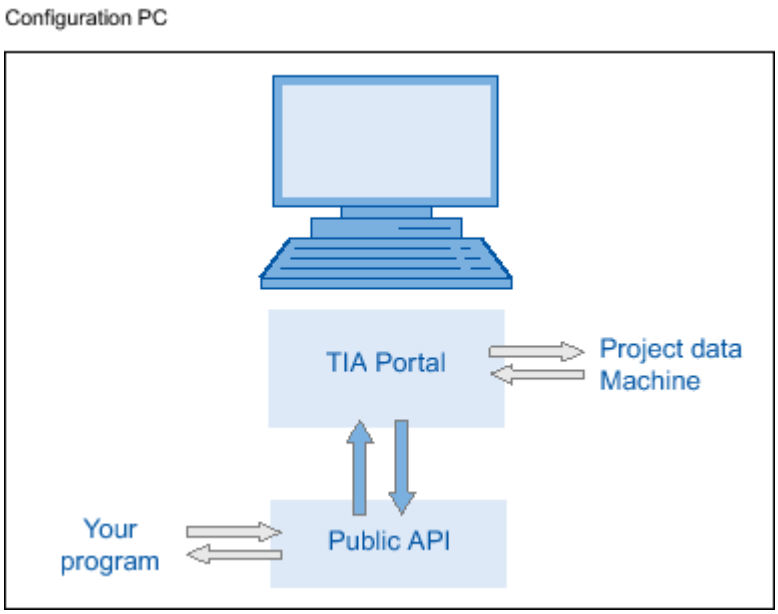
  Note

  You have to develop an executable program "Your Program 2" for PC2, such as "program2.exe". The TIA Portal runs with this program in the background.

  Import and export of XML files takes place exclusively via the TIA Portal Openness API.

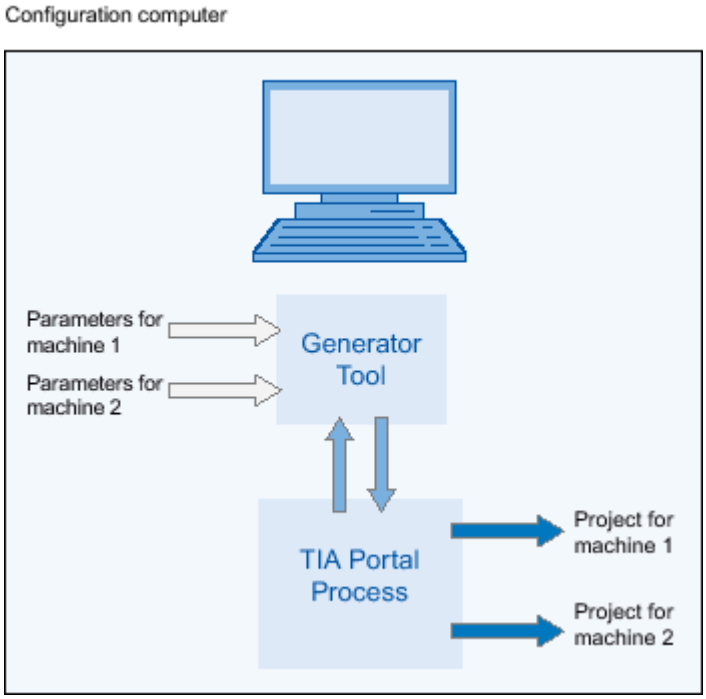
- You can archive exchanged files for verification purposes.
- Exchanged data can be processed at different locations and times.

Application and the TIA Portal are on the same computer



- Your program launches the TIA Portal either with or without the user interface. Your program opens, saves and/or closes a project. The program can also connect to a running TIA Portal.
- You can then use the TIA Portal functionality to request, generate and modify project data or to initiate import or export processes.
- The data is created under control of the TIA Portal processing and stored in the project data.

Typical application in modular mechanical engineering



- An efficient automation system is to be applied to similar machines.
- A project is available in the TIA Portal, which contains the components of all machine variants.

- The Generator tool controls the creation of the project for a specific machine variant.
- The Generator tool obtains the defaults by reading in the parameters for the requested machine variant.
- The Generator tool filters out the relevant elements from the overall TIA Portal project, modifies them if necessary and generates the requested machine project.

1.3.2 Openness tasks

1.3.2.1 Introduction

Introduction

TIA Portal Openness describes open interfaces for engineering with the TIA Portal. You will find further information about "TIA Portal Openness - Efficient generation of program code using code generators" in the SIEMENS YouTube channel.

You automate the engineering with TIA Portal Openness by controlling the TIA Portal externally from a program you have created.

You can perform the following actions with TIA Portal Openness:

- Create project data
- Modify projects and project data
- Delete project data
- Read in project data
- Make projects and project data available for other applications.

Note

Siemens is not liable for and does not guarantee the compatibility of the data and information transported via these interfaces with third-party software.

We expressly point out that improper use of the interfaces can result in data loss or production downtimes.

Note

The code snippets contained in this documentation are written in C# syntax.

Due to the shortness of the used code snippets the error handling description has been shortened as well.

Application

The TIA Portal Openness interface is used to do the following:

- Provide project data.

- Access the TIA Portal process.
- Use project data.

Using defaults from the field of automation engineering

- By importing externally generated data
- By remote control of the TIA Portal for generating projects

Providing project data of the TIA Portal for external applications

- By exporting project data

Ensuring competitive advantages through efficient engineering

- You do not have to configure existing engineering data in the TIA Portal.
- Automated engineering processes replace manual engineering.
- Low engineering costs strengthen the bidding position as compared to the competition.

Working together on project data

- Test routines and bulk data processing can take place in parallel to the ongoing configuration.

See also

Configurations

1.3.2.2 Applications

Introduction

TIA Portal Openness provides you with various ways to access the TIA Portal and offers a selection of functions for defined tasks.

You access the following areas of the TIA Portal by using the TIA Portal Openness API interface :

- Portal data
- Project data
- PLC data
- HMI data
- Drive data

Accessing the TIA Portal

TIA Portal Openness offers various ways to access the TIA Portal. You create an external TIA Portal instance in the process either with or without UI. You can also access ongoing TIA Portal processes at the same time.

Accessing projects and project data

When accessing projects and project data, you mainly use TIA Portal Openness for the following tasks:

- Close, open and save the project
- Enumerate and query objects
- Create objects
- Delete objects

1.3.2.3 Export/import

Introduction

TIA Portal Openness supports the import and export of project data by means of XML files. The import/export function supports external configuration of existing engineering data. You use this function to make the engineering process effective and free of error.

Application

You use the import/export function for the following purposes:

- Data exchange
- Copying parts of a project
- External processing of configuration data, for example, for bulk data operations using find and replace
- External processing of configuration data for new projects based on existing configurations
- Importing externally-created configuration data, for example, text lists and tags
- Providing project data for external applications

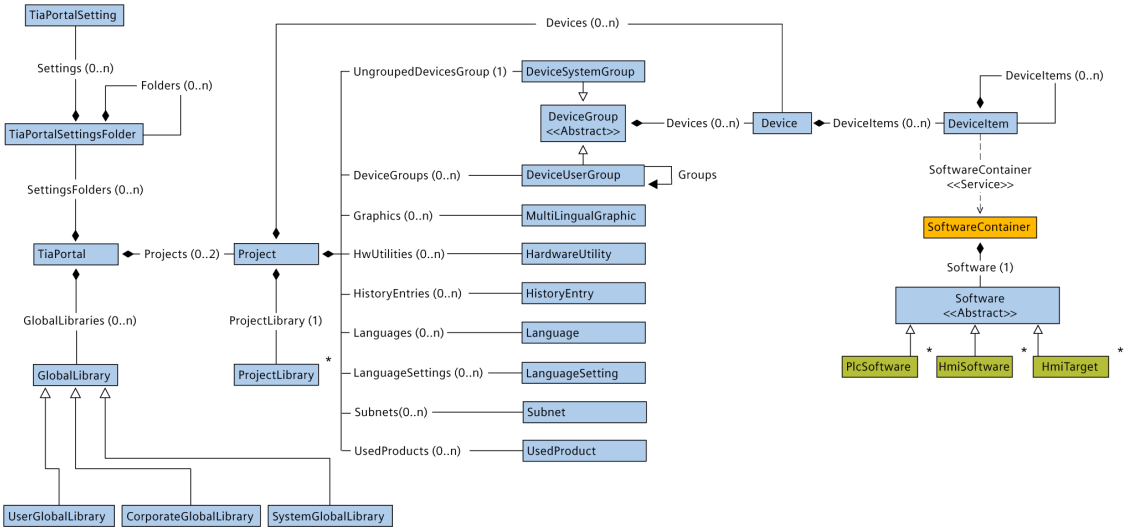
1.4 TIA Portal Openness API

1.4.1 TIA Portal Openness object

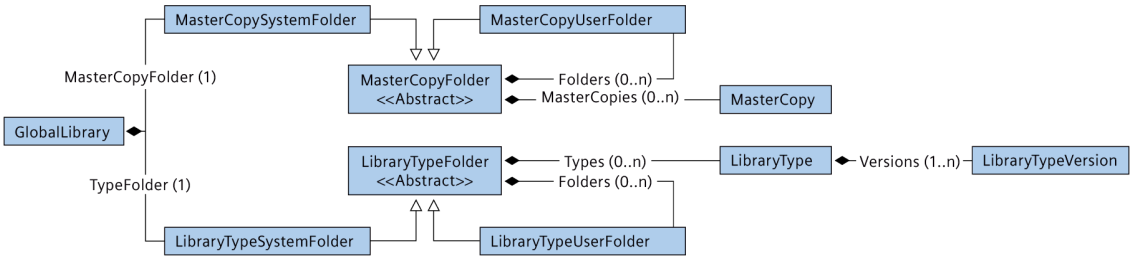
1.4.1.1 TIA Portal Openness object model

Overview

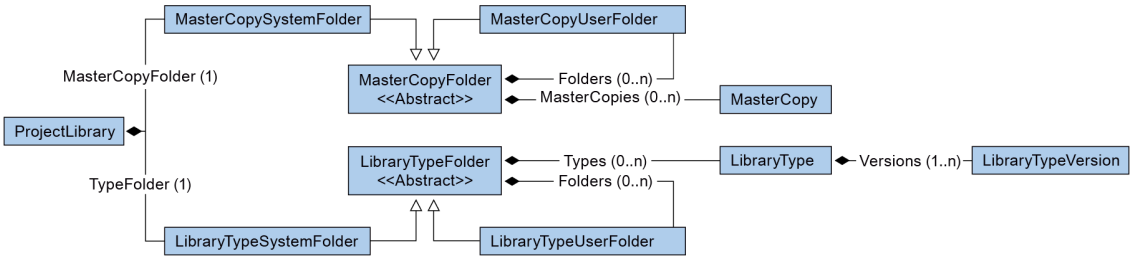
The following diagram describes the highest level of the TIA Portal Openness object model:



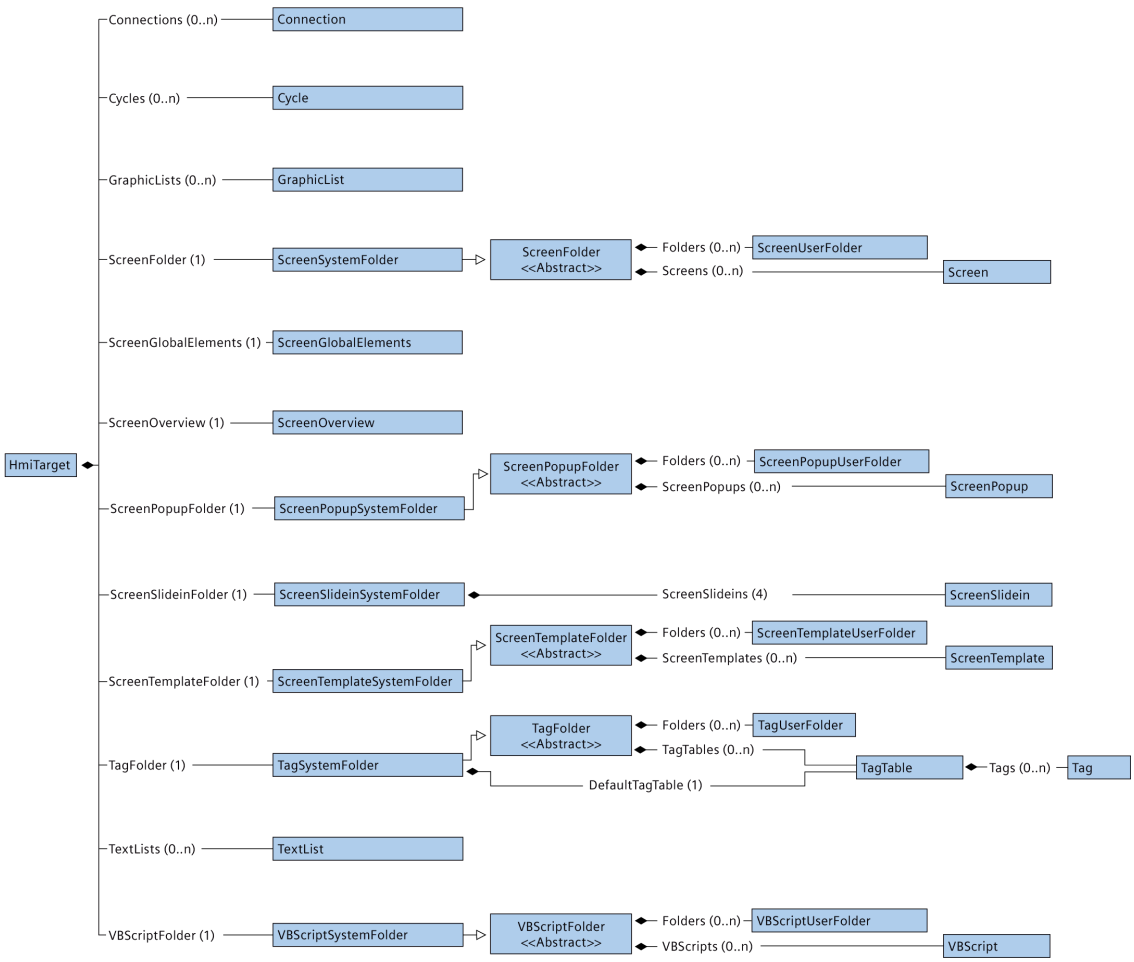
The following diagram describes the objects which are located under GlobalLibrary.



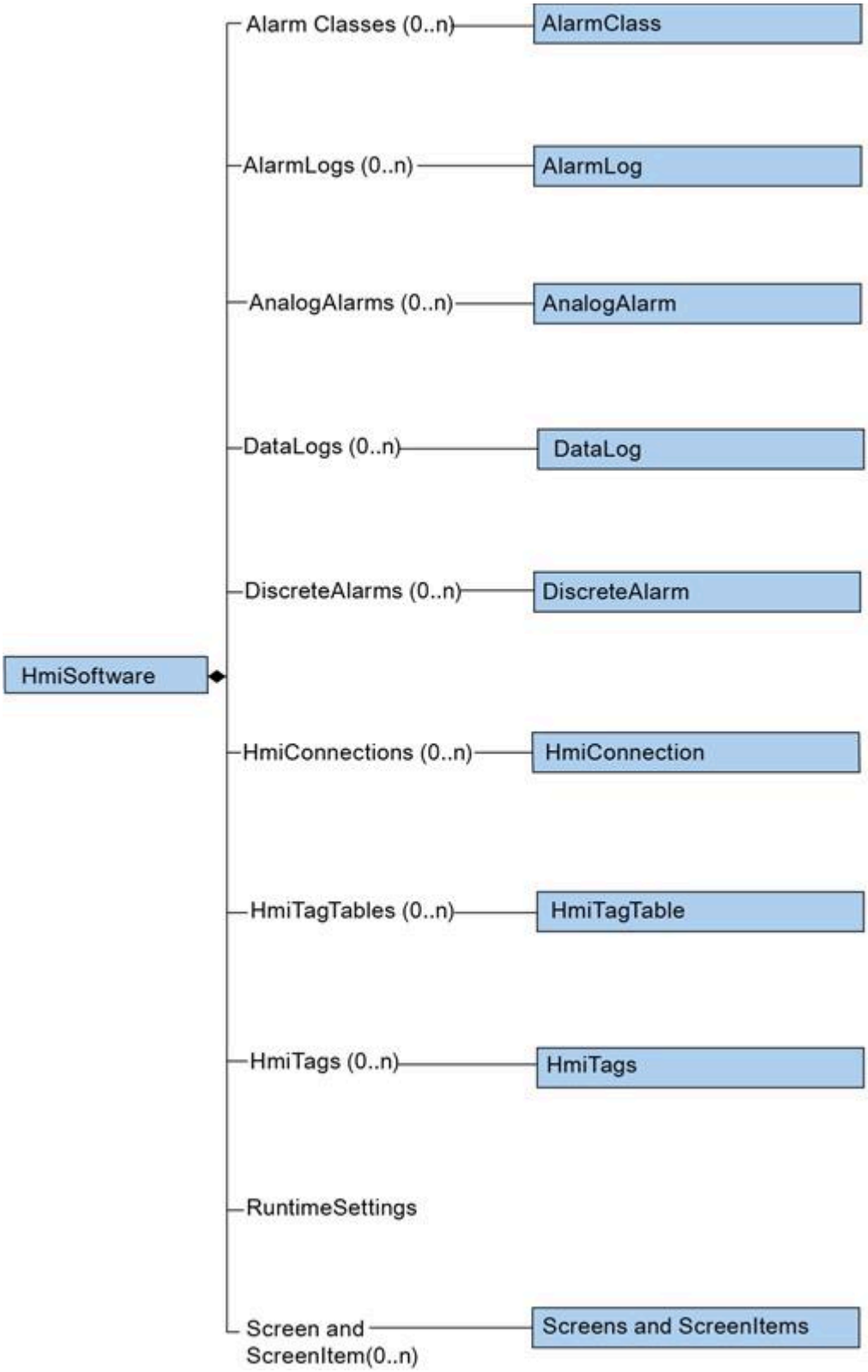
The following diagram describes the objects which are located under ProjectLibrary.



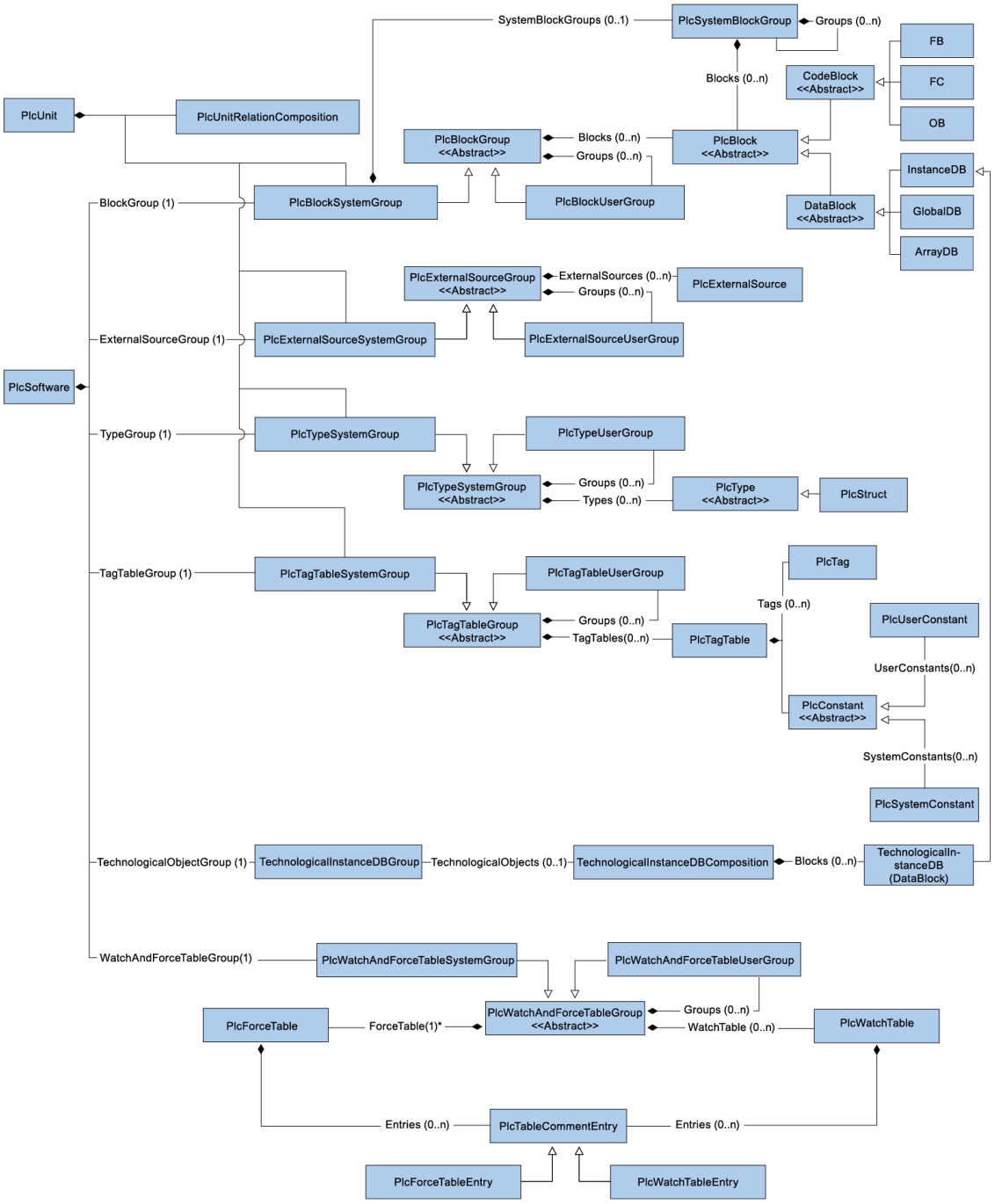
The following diagram describes the objects which are located under HmiTarget.



The following diagram describes the object which are located under HmiSoftware.



The following diagram describes the objects which are located under PlcSoftware.



Note

*The Force Table must be in PlcWatch and ForceTableSystemGroup

Access to objects in lists

You have the following options for addressing an object in a list:

- Address via the index. The counting within the lists starts with `0`.

- Use `Find` method.
  Use this method to address an object via its name. You can use this method for an composition or list. The `Find` method is not recursive.

  Example:

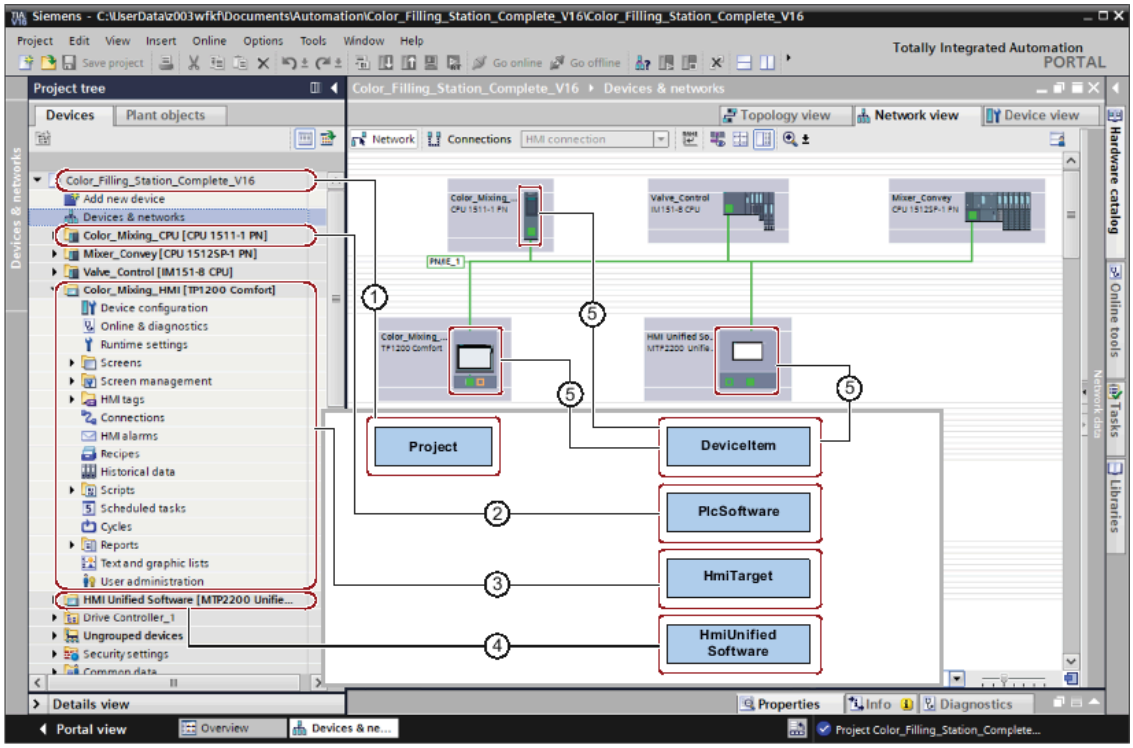
  ```
  ScreenComposition screens = folder.Screens;

  Screen screen = screens.Find("myScreen");
  ```

- Use symbolic names.

Relationship between TIA Portal and TIA Portal Openness object model

The figure below shows the relationship between the object model and a project in the TIA Portal:



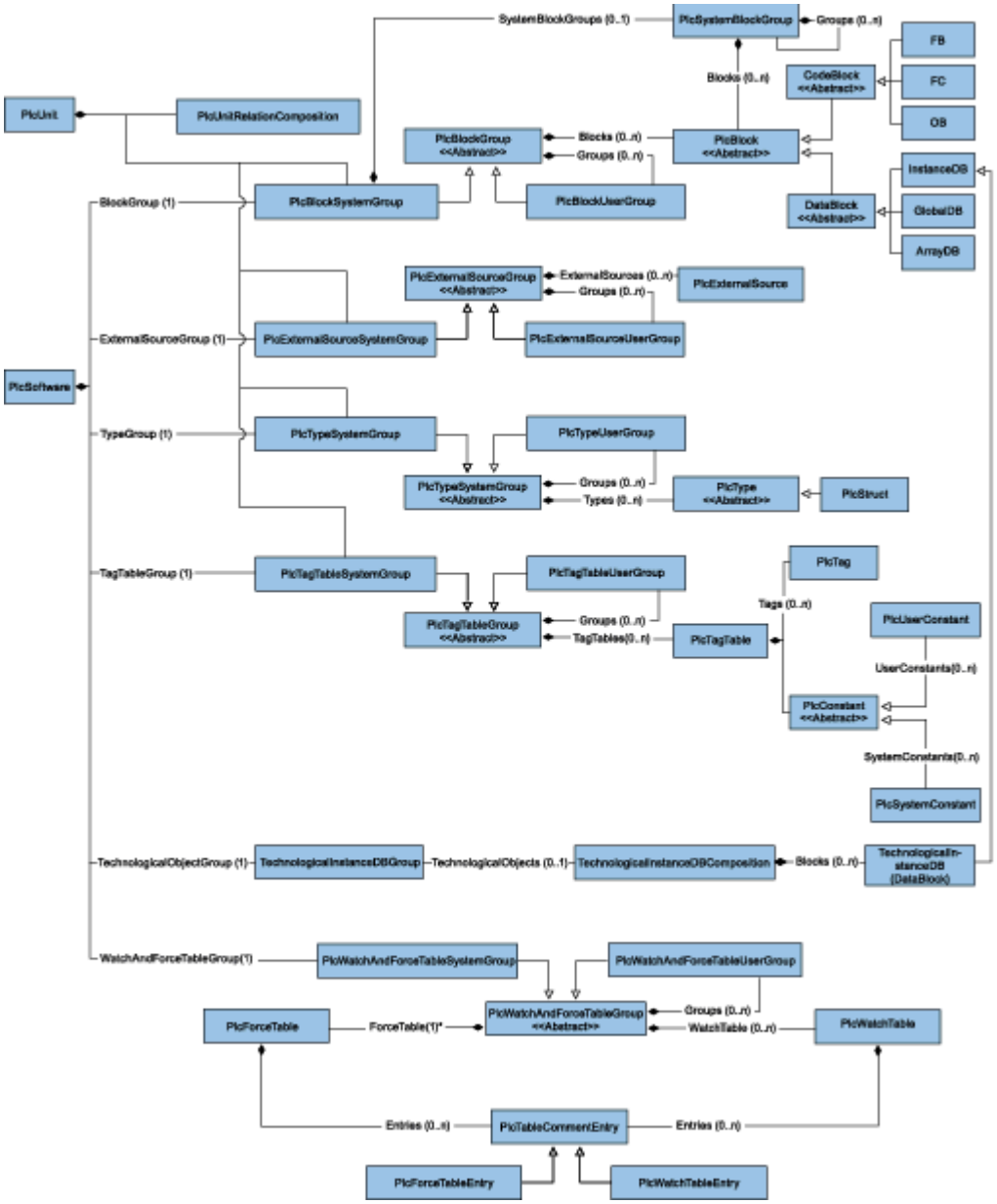
See also

Blocks and types of the TIA Portal Openness object model
Hierarchy of hardware objects of the object model

1.4.1.2 Blocks and types of the TIA Portal Openness object model

Introduction

The following diagram describes the domain model of the PLCs to give an overview of the current modeling in TIA Portal Openness.



Note

* The Force Table must be in PlcWatch and ForceTableSystemGroup

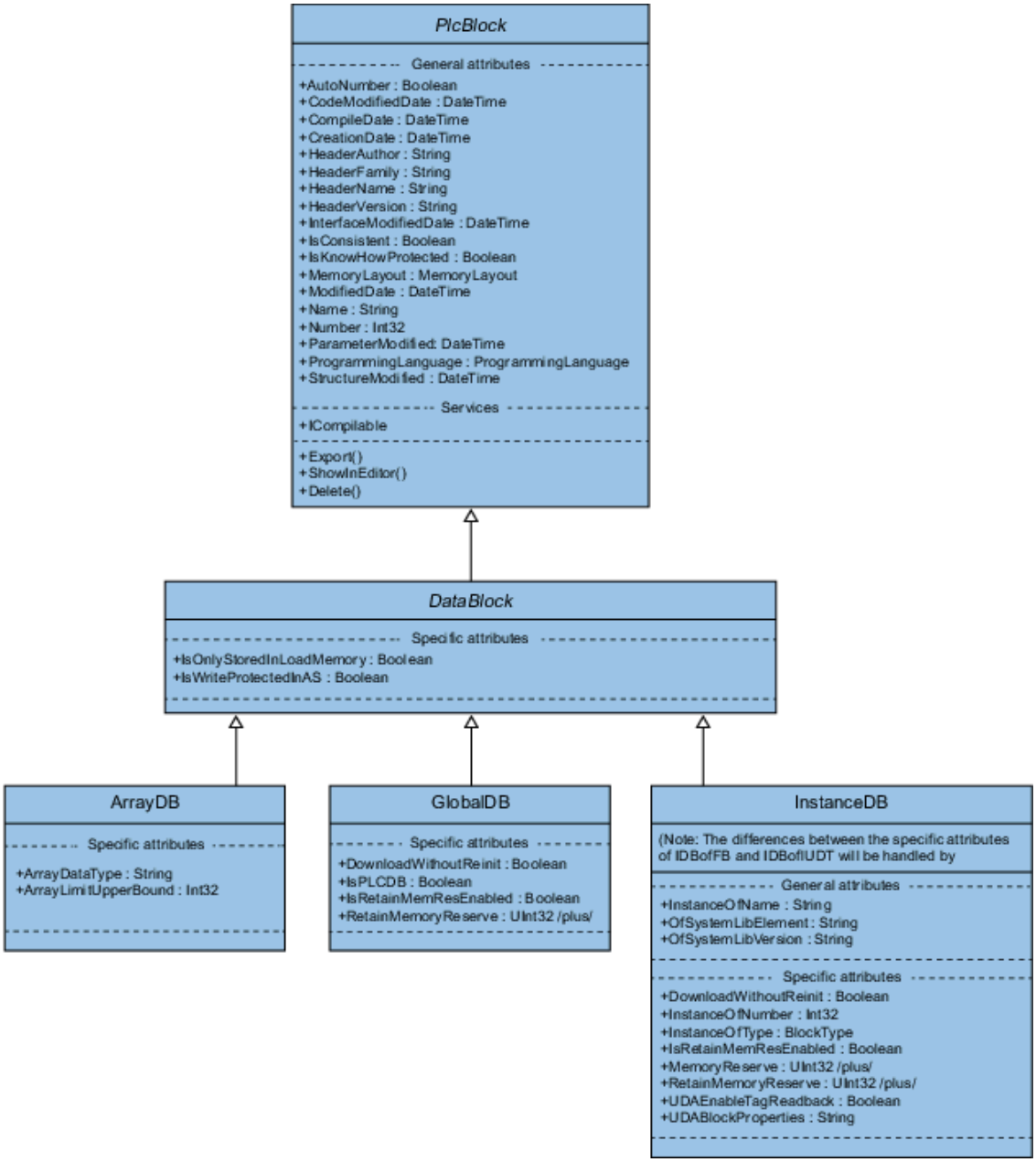
Representation of blocks and types in the TIA Portal Openness API

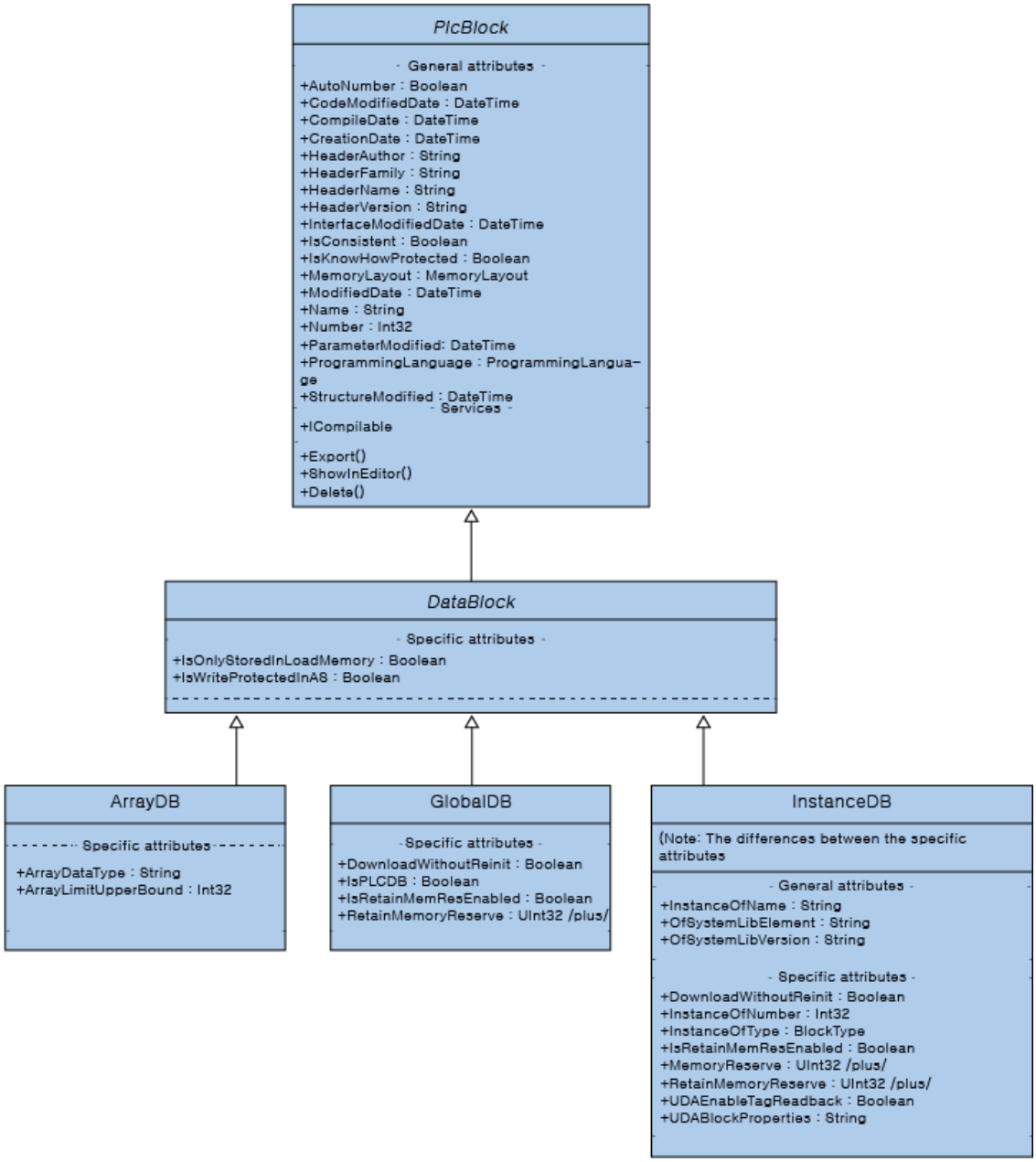
The simplified model part of blocks and for the structure is based on the attributes in the TIA Portal Openness API. These classes provide the export function and for blocks also the compile function.

Class diagrams

In the TIA Portal Openness object model all classes are defined as abstract which aren't directly instantiated.

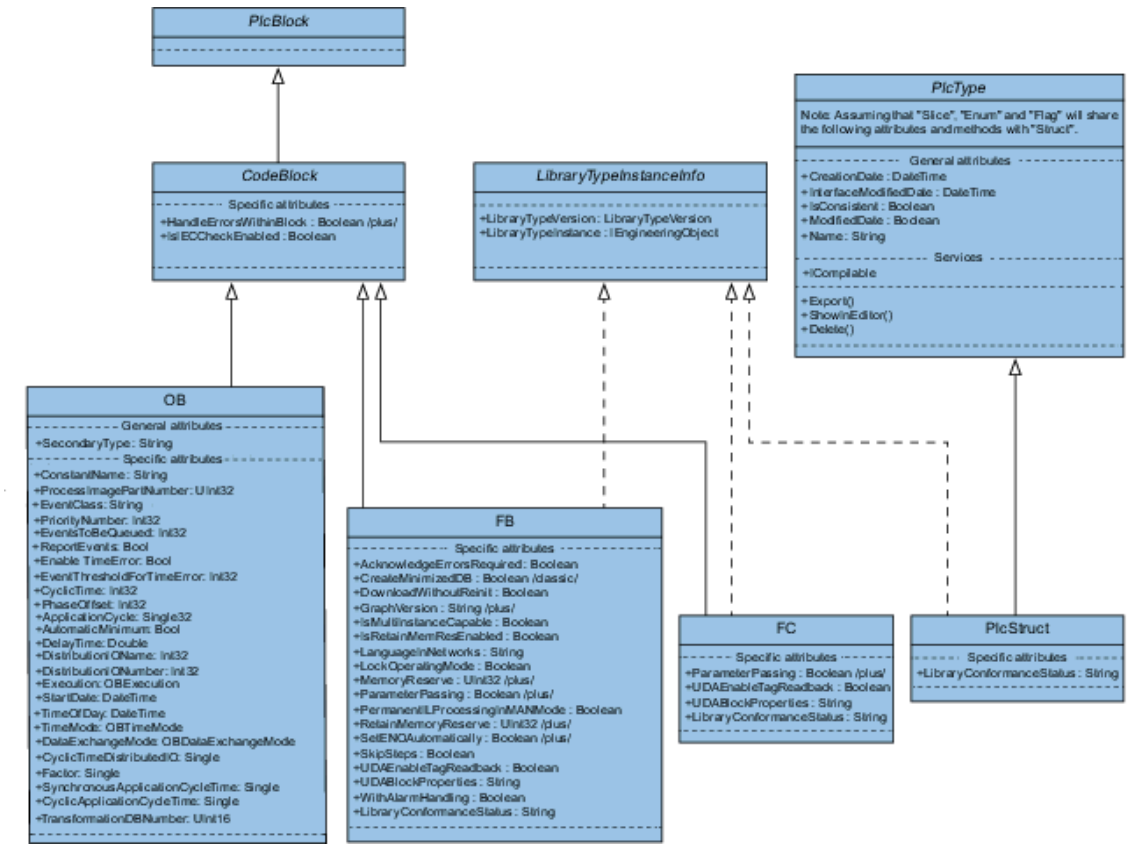
Data


PlcBlock
General attributes
+AutoNumber : Boolean
+CodeModifiedDate : DateTime
+CompileDate : DateTime
+CreationDate : DateTime
+HeaderAuthor : String
+HeaderFamily : String
+HeaderName : String
+HeaderVersion : String
+InterfaceModifiedDate : DateTime
+IsConsistent : Boolean
+IsKnowHowProtected : Boolean
+MemoryLayout : MemoryLayout
+ModifiedDate : DateTime
+Name : String
+Number : Int32
+ParameterModified: DateTime
+ProgrammingLanguage : ProgrammingLanguage
+StructureModified : DateTime
Services
+ICompilable
+Export()
+ShowInEditor()
+Delete()
DataBlock
Specific attributes
+IsOnlyStoredInLoadMemory : Boolean
+IsWriteProtectedInAS : Boolean
ArrayDB
Specific attributes
+ArrayDataType : String
+ArrayLimitUpperBound : Int32
GlobalDB
Specific attributes
+DownloadWithoutReinit : Boolean
+IsPLCDB : Boolean
+IsRetainMemResEnabled : Boolean
+RetainMemoryReserve : UInt32 /plus/
InstanceDB
(Note: The differences between the specific attributes of IDBofFB and IDBofUDT will be handled by
General attributes
+InstanceOfName : String
+OfSystemLibElement : String
+OfSystemLibVersion : String
Specific attributes
+DownloadWithoutReinit : Boolean
+InstanceOfNumber : Int32
+InstanceOfType : BlockType
+IsRetainMemResEnabled : Boolean
+MemoryReserve : UInt32 /plus/
+RetainMemoryReserve : UInt32 /plus/
+UDAEnableTagReadback : Boolean
+UDABlockProperties : String



Representation of block diagram

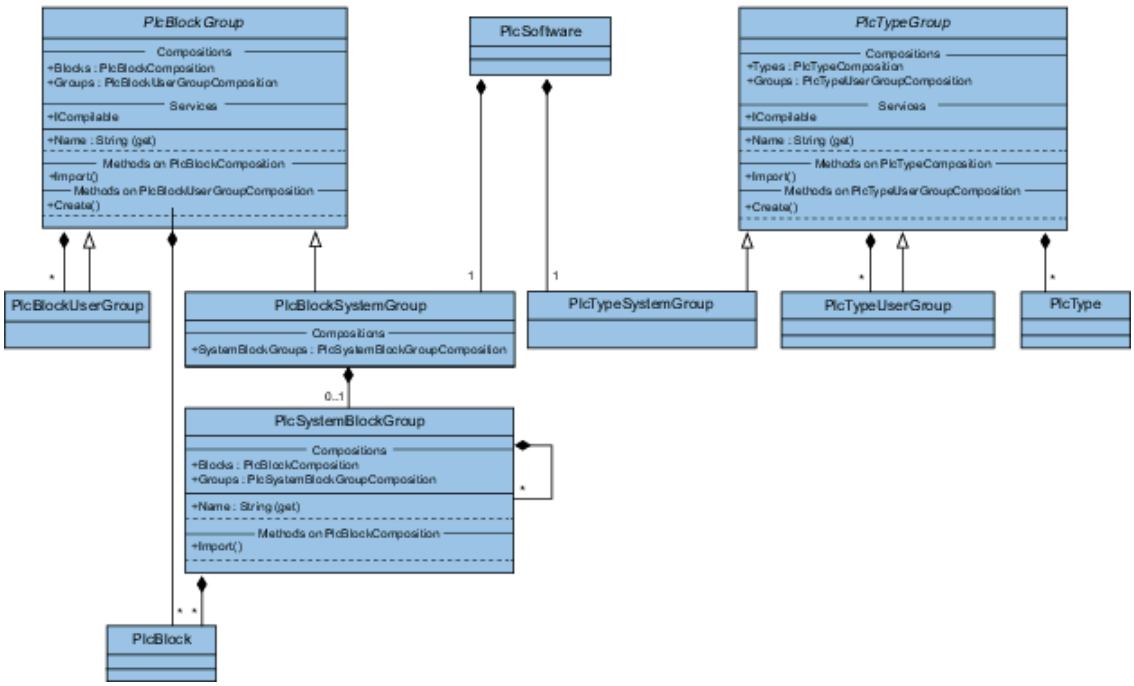
Code and Type



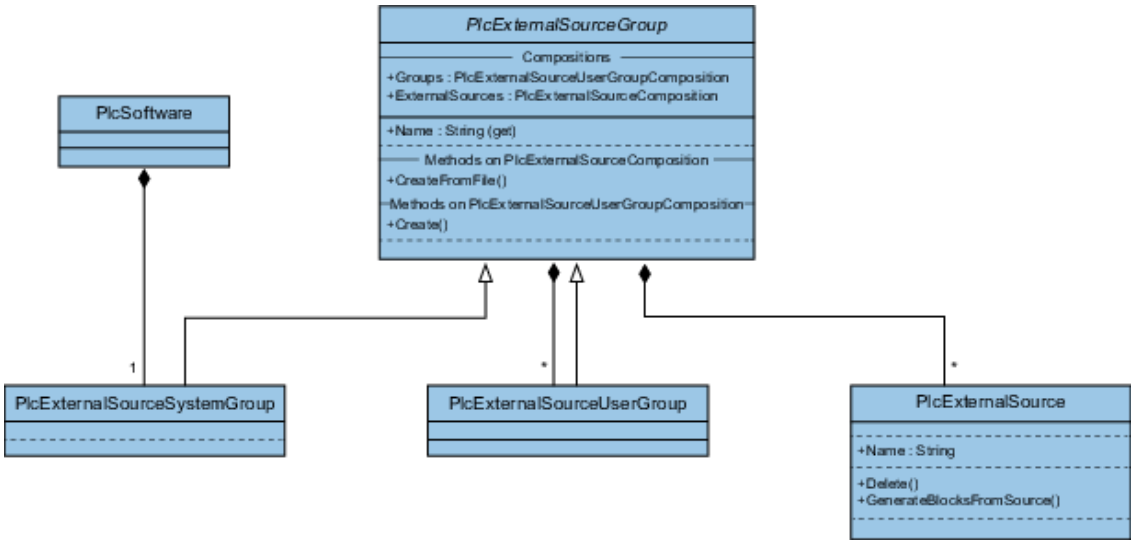
Representation of groups for blocks and types in the TIA Portal Openness API

The two high level groups "PlcBlocks" ("Program blocks" in the GUI of TIA Portal) and "PlcTypes" ("Plc data types" in the GUI of TIA Portal) contain blocks and type definitions. These groups provide the import and the compile functions for blocks. Due to the fact that most of the methods of functionalities of the groups are only achievable via collections, there is an "embedded" or "compacted" representation of the collections and their methods at the "host" classes.

Blocks and Types



External Sources



See also

TIA Portal Openness object model
Hierarchy of hardware objects of the object model

1.4.1.3 Hierarchy of hardware objects of the object model

Relation between the visible elements in the TIA Portal and the modeled elements in the object model

Hardware object	Explanation
Device (Device)	The container object for a central or distributed configuration.
Device item (DeviceItem)	Each device item object has a container object. The logical relation is "Items".

To understand the hierarchy relationship of hardware objects between what is displayed in the TIA Portal view and what is modeled in the TIA Portal Openness, it is necessary to consider some aspects of the hierarchy of hardware objects.

Example: A HardwareObject has the items link. The Device extends the HardwareObject and has the UnpluggedItems link. The Device is the container object for a station.

The DeviceItem extends the HardwareObject and has the Container link. Every plugged DeviceItem object has got a container object which is a HardwareObject. The reverse relation is Items.

An unplugged DeviceItem object that does not have a container is found via the link UnpluggedItems of the Device.

The container relation is comparable to the relation of the modules for the device item objects.

Example: A device includes one or more racks. A rack includes modules. A module includes submodules.

This is the relation similar to the representation in the network view and device view of the TIA Portal. The "PositionNumber" attribute of a device item is unique in the items area, within a container.

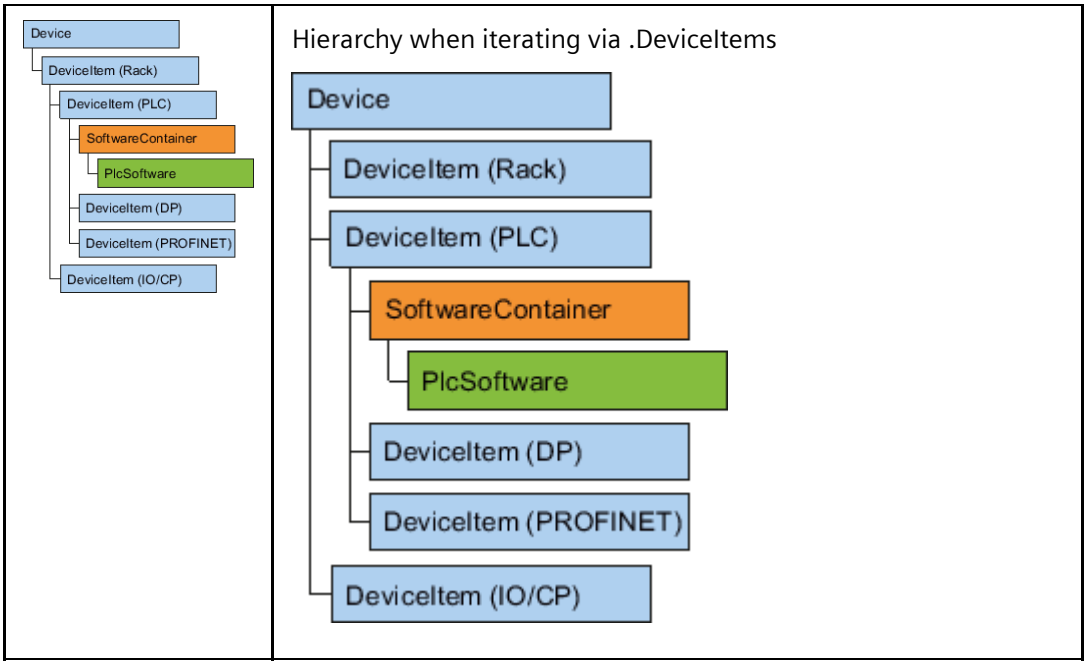
The parent-child relation between device item objects is a purely logical relation in the object model. A child cannot exist without its parents.

- If a submodule is modeled as part of a module (child), the submodule cannot be removed without the module.
- A module or submodule that can be added and removed is a child of the device object

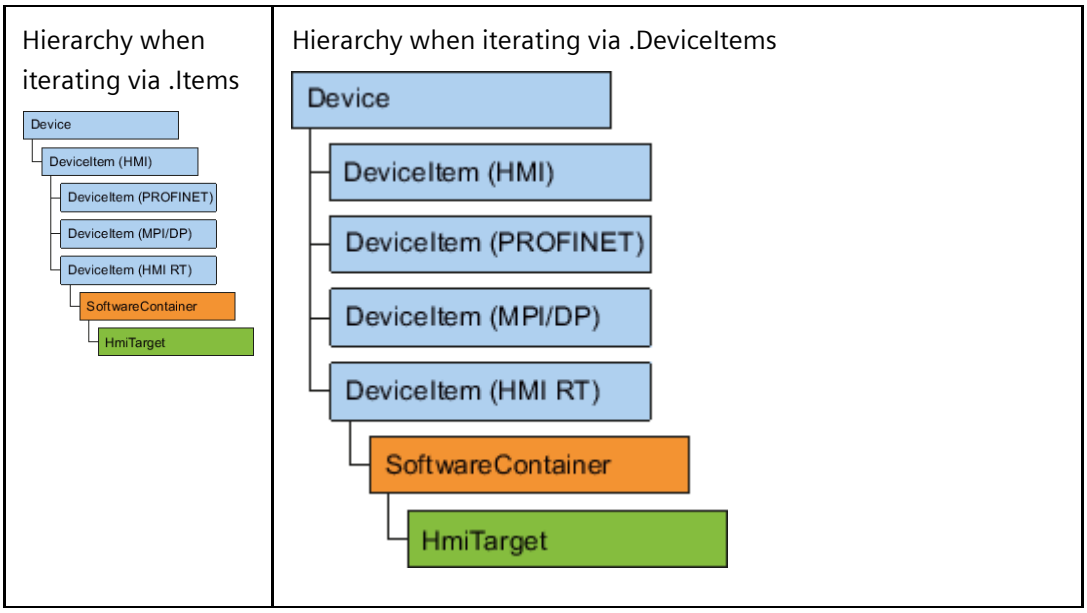
Hierarchy relationships of PLC devices

The diagrams below show the hierarchy relationship between devices, and device items of PLC and HMI devices.

Hierarchy when iterating via .Items	



Hierarchy relationships of HMI devices



See also

TIA Portal Openness object model
Blocks and types of the TIA Portal Openness object model

1.4.1.4 Object list

Introduction

The following tables show the available objects up to and including Runtime Advanced, and indicate whether these objects are supported by TIA Portal Openness.

Neither Runtime Professional nor device proxy files are supported by TIA Portal Openness in WinCC.

Objects

You can control the following project data depending on which HMI device you are using:

Screens

Object	Basic Panels	Comfort Panels	Mobile Panels	RT Advanced
Screen	Yes	Yes	Yes	Yes
Global screen	Yes	Yes	Yes	Yes
Templates	Yes	Yes	Yes	Yes
Permanent area	No	Yes	Yes	Yes
Pop-up screen	No	Yes	Yes	Yes
Slide-in screen	No	Yes	Yes	Yes

Screen objects

Object	Basic Panels	Comfort Panels	Mobile Panels	RT Advanced
Line	Yes	Yes	Yes	Yes
Polyline	No	Yes	Yes	Yes
Polygon	No	Yes	Yes	Yes
Ellipse	Yes	Yes	Yes	Yes
Ellipse segment	No	No	No	No
Circle segment	No	No	No	No
Elliptical arc	No	No	No	No
Camera view	No	No	No	No

Circular arc	No	No	No	No
Circle	Yes	Yes	Yes	Yes
PDF view	No	No	No	No
Rectangle	Yes	Yes	Yes	Yes
Connector	No	No	No	No
Text field	Yes	Yes	Yes	Yes
Graphic view	Yes	Yes	Yes	Yes
Pipe	No	No	No	No
Double T-piece	No	No	No	No
T-piece	No	No	No	No
Pipe bends	No	No	No	No
I/O field	Yes	Yes	Yes	Yes
Date/time field	Yes	Yes	Yes	Yes
Graphic I/O field	Yes	Yes	Yes	Yes
Editable text field	No	No	No	No
List box	No	No	No	No
Combo box	No	No	No	No
Button	Yes	Yes	Yes	Yes
Round button	No	No	No	No
Illuminated button	No	No	Yes	No
Switch	Yes	Yes	Yes	Yes
Symbolic I/O field	Yes	Yes	Yes	Yes

Key-operated switch	No	No	Yes	No
Bar	Yes	Yes	Yes	Yes
Symbol library	No	Yes	Yes	Yes
Slider	No	Yes	Yes	Yes
Scroll bar	No	No	No	No
Check box	No	No	No	No
Option buttons	No	No	No	No
Gauge	No	Yes	Yes	Yes
Clock	No	Yes	Yes	Yes
Memory space view	No	No	No	No
Function keys	Yes	Yes	Yes	Yes
Faceplate instances	No	Yes	Yes	Yes
Screen window	No	No	No	No
User view	Yes	Yes	Yes	Yes
HTML Browser	No	No	No	No
Print job/script diagnostics	No	No	No	No
Recipe view	No	No	No	No
Alarm view	No	No	No	No
Alarm indicator	No	No	No	No
Alarm window	No	No	No	No
Criteria analysis view	No	Yes [1)]	Yes	Yes
ProDiag overview	No	Yes [1)]	Yes	Yes

GRAPH overview	No	Yes [1]	Yes	Yes
PLC code view	No	Yes [1]	Yes	Yes
f(x) trend view	No	No	No	No
f(t) trend view	No	No	No	No
Table view	No	No	No	No
Value table	No	No	No	No
Media Player	No	No	No	No
Channel diagnostics	No	No	No	No
WLAN reception	No	No	No	No
Zone name	No	No	No	No
Zone signal	No	No	No	No
Effective range name	No	No	No	No
Effective range name (RFID)	No	No	No	No
Effective range signal	No	No	No	No
Charge condition	No	No	No	No
Handwheel	No	No	Yes	No
Help indicator	No	No	No	No
Sm@rtClient view	No	No	No	No
Status/Force	No	No	No	No
System diagnostic view	No	No	No	No
System diagnostic window	No	No	No	No

1) Only Mobile Panels with the device version greater than 12.0.0.0 support this screen object

Dynamic

Object	Basic Panels	Comfort Panels	Mobile Panels	RT Advanced
Display	Yes	Yes	Yes	Yes
Operability	No	Yes	Yes	Yes
Visibility	Yes	Yes	Yes	Yes
Movements	Yes	Yes	Yes	Yes

Additional objects

Object	Basic Panels	Comfort Panels	Mobile Panels	RT Advanced
Groups	Yes	Yes	Yes	Yes
Soft keys	Yes	Yes	Yes	Yes
Cycles	Yes	Yes	Yes	Yes
VB scripts	No	Yes	Yes	Yes
Function lists	Yes	Yes	Yes	Yes
Project graphics	Yes	Yes	Yes	Yes

Tags

Object	Basic Panels	Comfort Panels	Mobile Panels	RT Advanced
Multiplex tags	Yes	Yes	Yes	Yes
Arrays	Yes	Yes	Yes	Yes
User data types	Yes	Yes	Yes	Yes
Internal	No	Yes	Yes	Yes

Points of use of elementary data types	Yes	Yes	Yes	Yes
Points of use of user data types	Yes	Yes	Yes	Yes
Points of use of arrays	Yes	Yes	Yes	Yes

TIA Portal Openness also supports all value ranges which are supported by the communication drivers.

Connections

TIA Portal Openness supports non-integrated connections that are also supported by the respective HMI devices. You can find additional information in the online help for the TIA Portal under "Process visualization > Controller communication > Device-dependent".

Lists

Object	Basic Panels	Comfort Panels	Mobile Panels	RT Advanced
Text lists	Yes	Yes	Yes	Yes
Graphic lists	Yes	Yes	Yes	Yes

Texts

Object	Basic Panels	Comfort Panels	Mobile Panels	RT Advanced
Multilingual texts	Yes	Yes	Yes	Yes
Formatted texts and their instances	No	Yes	Yes	Yes

1.4.2 General functions

1.4.2.1 Introduction

Overview

TIA Portal Openness supports a selection of functions for defined tasks that you can call outside the TIA Portal by means of the TIA Portal Openness API.

Note

If a previous version of TIA Portal Openness is already installed, the current version will be installed side by side.

You are provided with an overview of the typical programming steps in the sections below. You can learn how the individual code sections interact and how to integrate the respective functions into a complete program. You also get an overview of the code components that have to be adapted for each task.

Example program

The individual programming steps are explained using the "Creating API access in a console application" function as an example. You integrate the provided functions in this program code and adapt the respective code components for this task.

Functions

The section below lists the functions for defined tasks that you can call with TIA Portal Openness outside the TIA Portal.

See also

Applications
Object list

1.4.2.2 Information about installed TIA Portal Openness versions

Each installed TIA Portal version provides information about available, compatible TIA Portal Openness versions in the Windows registry. This enables TIA Portal Openness applications to automatically start or connect to matching TIA Portal instances.

The relevant TIA Portal Openness registry keys are in the following path in the Windows registry:

HKEY_LOCAL_MACHINE\SOFTWARE\Siemens\Automation\Openness

The registry key “Openness” contains the following subkeys and values:

- Version-independent subkey “AllowList” for the TIA Portal Openness firewall (only from TIA Portal V21 onwards)
  - HKEY_LOCAL_MACHINE\SOFTWARE\Siemens\Automation\Openness\AllowList
  - See also TIA Portal Openness firewall
- Subkeys for each installed TIA Portal version: “xx.xx” (technical version number)
  - HKEY_LOCAL_MACHINE\SOFTWARE\Siemens\Automation\Openness\xx.xx
  - The string value “PortalVersion” for a display string of that TIA Portal version (only from TIA Portal V21 onwards)
- Subkey “PublicAPI”
  - HKEY_LOCAL_MACHINE\SOFTWARE\Siemens\Automation\Openness\xx.xx\PublicAPI

- Subkeys for each provided TIA Portal Openness version in that specific TIA Portal version: "xx.xx.xx.xx" (technical version number)
  - HKEY_LOCAL_MACHINE\SOFTWARE\Siemens\Automation\Openness\xx.xx\PublicAPI\xx.xx.xx.xx
  - The string value "EngineeringVersion" for a display string of that TIA Portal Openness version
- Subkeys for each supported .NET target framework version in that specific TIA Portal Openness version using the Microsoft Target Framework Moniker (TFM) which is a standardized token format for specifying the target framework of a .NET library (only from TIA Portal V21 onwards): "netxxx"
  - HKEY_LOCAL_MACHINE\SOFTWARE\Siemens\Automation\Openness\xx.xx\PublicAPI\xx.xx.xx.xx\netxx
  - The string value "AssemblyVersion" which the Microsoft CLR uses to load the exact version
  - The string value "PublicKeyToken" as part of the strong-named assembly to create a unique identity to prevent assembly conflicts
  - The string value "Siemens.Engineering.Base" pointing to the custom installation path

Note

Regardless which TIA Portal products are installed, the Windows registry only provides the installation path to the base assembly "Siemens.Engineering.Base".

See also

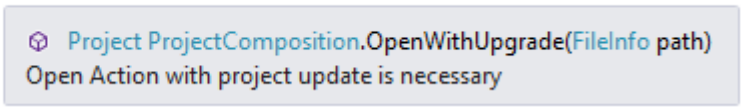
Connecting to the TIA Portal

1.4.2.3 TIA Portal Openness IntelliSense support

Application

The Intellisense support of TIA Portal Openness helps you at available attributes or methods via tooltip information. It could contain information about the number, names and types of the required parameters. At the following example the bold parameter in the first line indicates the next parameter that is required as you type the function.

```
project = tiaPortal.Projects.OpenWithUpgrade(projectInfo);
```



You can manually invoke Parameter Info by clicking Edit IntelliSense/Parameter Info, typing CTRL+SHIFT+SPACE, or clicking the Parameter Info button on the editor toolbar.

1.4.2.4 Standard libraries

Insert the following namespace statements at the beginning of the relevant code example to ensure that the code examples work:



```
Copies the following program code to clipboard.
using System;
using System.Collections.Generic;
using System.IO;
using Siemens.Engineering;
using Siemens.Engineering.Cax;
using Siemens.Engineering.Compiler;
using Siemens.Engineering.Compare;
using Siemens.Engineering.Download;
using Siemens.Engineering.Hmi;
using Siemens.Engineering.Hmi.Cycle;
using Siemens.Engineering.Hmi.Communication;
using Siemens.Engineering.Hmi.Globalization;
using Siemens.Engineering.Hmi.RuntimeScripting;
using Siemens.Engineering.Hmi.Screen;
using Siemens.Engineering.Hmi.Tag;
using Siemens.Engineering.Hmi.TextGraphicList;
using Siemens.Engineering.HW;
using Siemens.Engineering.HW.Extensions;
using Siemens.Engineering.HW.Features;
using Siemens.Engineering.HW.Utilities;
using Siemens.Engineering.Library;
using Siemens.Engineering.Library.MasterCopies;
using Siemens.Engineering.Library.Types;
using Siemens.Engineering.SW;
using Siemens.Engineering.SW.Blocks;
using Siemens.Engineering.SW.ExternalSources;
using Siemens.Engineering.SW.Tags;
```

```
using Siemens.Engineering.SW.TechnologicalObjects;
using Siemens.Engineering.SW.TechnologicalObjects.Motion;
using Siemens.Engineering.SW.Types;
using Siemens.Engineering.Upload;
```

1.4.2.5 Use of the code examples

Structure of the code-snippets

Each code-snippet in this documentation is implemented as a function without return value with an object reference as transfer parameter. Disposing of objects is omitted for the sake of readability. Objects of the TIA Portal are addressed by their name using the `Find` method.



```
Copies the following program code to clipboard.
//Deletes a single screen from a user folder or a system folder
private static void DeleteScreenFromFolder(HmiTarget hmiTarget)
{
    //The screen "MyScreen" will be deleted if it is existing in the folder "myScreenFolder".
    //If "myScreen" is stored in a subfolder of "myScreenFolder" it will not be deleted.
    string screenName = "MyScreen";
    ScreenUserFolder folder = hmiTarget.ScreenFolder.Folders.Find("myScreenFolder");
    ScreenComposition screens = folder.Screens;
    Screen screen = screens.Find(screenName);
    if (screen != null)
    {
        screen.Delete();
    }
}
```

You need the following to execute this code-snippet:

- A WinCC project with an HMI device that includes a group with at least one screen.
- A function that instantiates the HMI device.

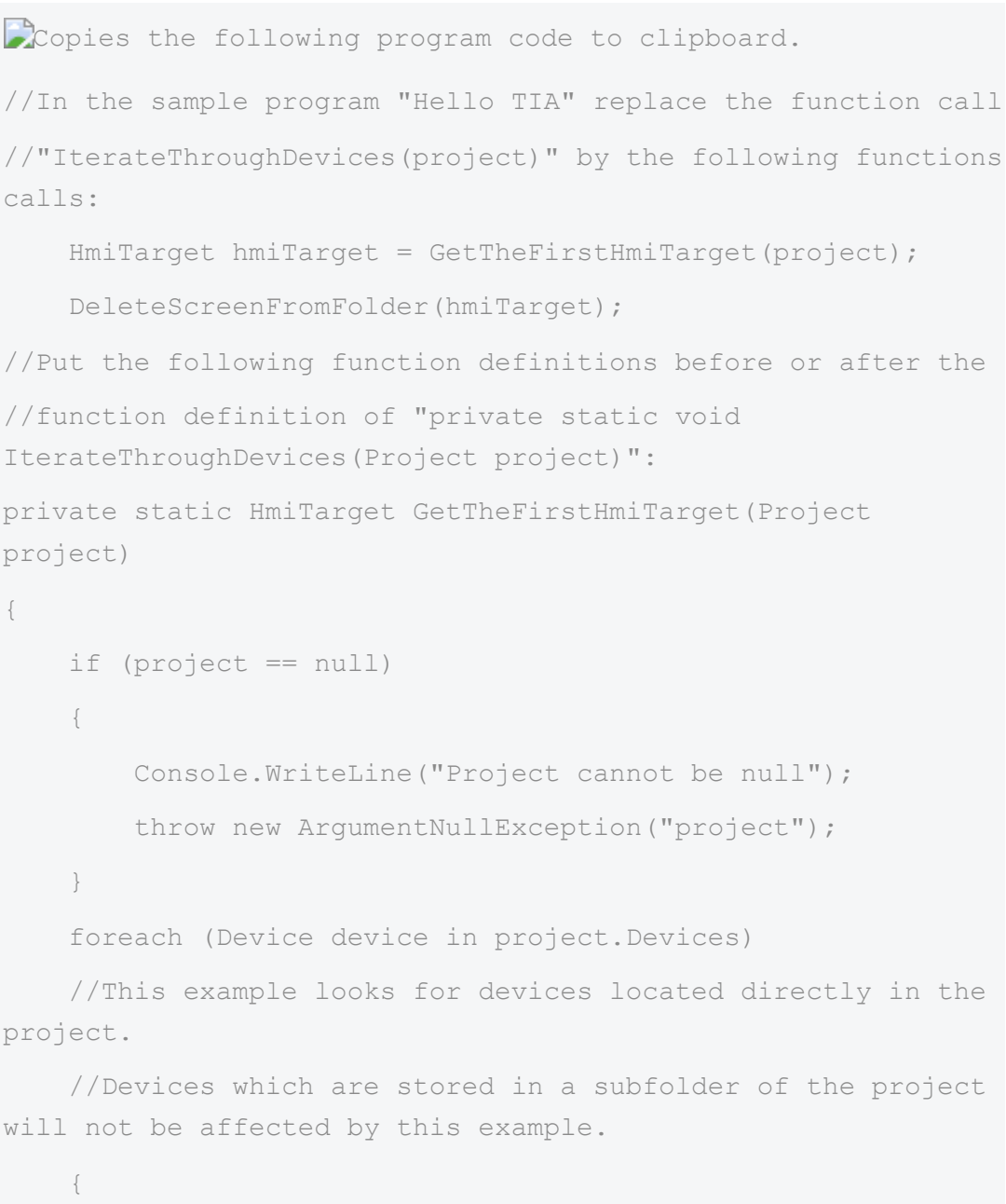
Note

When you specify directory paths, use the absolute directory path, for example, "C:/path/file.txt".

Relative directory paths are only allowed in the XML files for import and export, for example, "file.txt" or "C:/path01/.../path02/file.txt".

Example for execution of the code-snippet

Use the following example to execute the code-snippet "DeleteScreenFromFolder" as part of the "Hello TIA" example program:



```
        foreach (DeviceItem deviceItem in device.DeviceItems)
        {
            DeviceItem deviceItemToGetService = deviceItem as
DeviceItem;
            SoftwareContainer container =
deviceItemToGetService.GetService<SoftwareContainer>();
            if (container != null)
            {
                HmiTarget hmi = container.Software as
HmiTarget;
                if (hmi != null)
                {
                    return hmi;
                }
            }
        }
    }
    return null;
}
//Deletes a single screen from a user folder or a system
folder
private static void DeleteScreenFromFolder(HmiTarget
hmiTarget)
{
    string screenName = "MyScreen";
    ScreenUserFolder folder =
hmiTarget.ScreenFolder.Folders.Find("myScreenFolder");
    ScreenComposition screens = folder.Screens;
    Screen screen = screens.Find(screenName);
    if (screen != null)
    {
        screen.Delete();
    }
```

```
}
```

1.4.2.6 Example program

Application example: Creating API access in an application

The following program code shows basic API access in a Console application.

See also: Connecting to the TIA Portal

Note

Enabling the assembly reference property “Copy Local” is not supported. “Copy Local: True” means copying the references to Siemens.Engineering assemblies to the output directory of the application.

Remedy: Set “Copy Local: False” for every Siemens.Engineering assembly after adding the references to the project and clean the output directory.

 Copies the following program code to clipboard.

```
// Assembly reference: Siemens.Engineering.Base.dll
using Siemens.Engineering;
using Siemens.Engineering.Compiler;
using Siemens.Engineering.HW;
using Siemens.Engineering.HW.Features;

// Assembly reference: Siemens.Engineering.Step7.dll
using Siemens.Engineering.SW;
using Siemens.Engineering.SW.Units;

// System
using System;
using System.Collections.Generic;
using System.IO;
using System.Reflection;

namespace HelloOpenness
{
internal static class Program
{
private const string OpennessFolder = @"C:\Program
Files\Siemens\Automation\Portal Vxx\PublicAPI\Vxx\netxxx";

static Program()
```

```
{
internal static class Program
{
private const string OpennessFolder = @"C:\Program
Files\Siemens\Automation\Portal Vxx\PublicAPI\Vxx\netxxx";

static Program()
{
// Register assembly resolver at the earliest stage in static
Program constructor
AppDomain.CurrentDomain.AssemblyResolve += OnAssemblyResolve;
}

private static Assembly OnAssemblyResolve(object sender,
ResolveEventArgs args)
{
AssemblyName requestedAssemblyName = new
AssemblyName(args.Name);

string filePath = Path.Combine(OpennessFolder,
string.Concat(requestedAssemblyName.Name, ".dll"));

// Only respond to requests for TIA Portal Openness; must
ignore missing files
if
(!requestedAssemblyName.Name.StartsWith("Siemens.Engineering.")
|| !File.Exists(filePath))
return null;

Assembly loadedAssembly = Assembly.LoadFrom(filePath);

// Check if details like version match the requested assembly
if (requestedAssemblyName.FullName !=
loadedAssembly.GetName().FullName)
throw new FileNotFoundException("TIA Portal Openness version
does not match", filePath);

return loadedAssembly;
}

public static void Main()
```

```
internal static class Program
{
private const string OpennessFolder = @"C:\Program
Files\Siemens\Automation\Portal Vxx\PublicAPI\Vxx\netxxx";

static Program()
{
// Register assembly resolver at the earliest stage in static
Program constructor
AppDomain.CurrentDomain.AssemblyResolve += OnAssemblyResolve;
}

private static Assembly OnAssemblyResolve(object sender,
ResolveEventArgs args)
{
AssemblyName requestedAssemblyName = new
AssemblyName(args.Name);

string filePath = Path.Combine(OpennessFolder,
string.Concat(requestedAssemblyName.Name, ".dll"));

// Only respond to requests for TIA Portal Openness; must
ignore missing files
if
(!requestedAssemblyName.Name.StartsWith("Siemens.Engineering.")
|| !File.Exists(filePath))
return null;

Assembly loadedAssembly = Assembly.LoadFrom(filePath);

// Check if details like version match the requested assembly
if (requestedAssemblyName.FullName !=
loadedAssembly.GetName().FullName)
throw new FileNotFoundException("TIA Portal Openness version
does not match", filePath);

return loadedAssembly;
}

public static void Main()
```